TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI TP. HỒ CHÍ MINH

KHOA CÔNG NGHỆ THÔNG TIN

Ngành: Khoa học Máy tính — Mã ngành: 8480101

KHÓA LUẬN TỐT NGHIỆP

HỆ THỐNG PHÂN TÍCH SẮC THÁI CẢM XÚC TỪ TRANSCRIPT TIẾNG VIỆT TRONG HỌP TRỰC TUYẾN

Người hướng dẫn: TS. Đặng Nhân Cách

Sinh viên thực hiện: **[CẦN ĐIỀN: Họ và tên sinh viên]**

MSSV: **[CẦN ĐIỀN: MSSV]**

Lớp: **[CẦN ĐIỀN: Tên lớp]**

*TP. Hồ Chí Minh, tháng **[CẦN ĐIỀN:]** năm **[CẦN ĐIỀN:]***

Lời Cam Đoan

Tôi xin cam đoan rằng khóa luận tốt nghiệp này là công trình nghiên cứu của riêng tôi dưới sự hướng dẫn của TS. Đặng Nhân Cách. Các kết quả trình bày trong khóa luận là trung thực, chưa được công bố trong bất kỳ công trình nghiên cứu nào khác.

Mọi tham khảo, trích dẫn trong khóa luận đều được ghi rõ nguồn gốc và tác giả theo quy định của nhà trường và tiêu chuẩn trích dẫn học thuật quốc tế (IEEE). Tôi hoàn toàn chịu trách nhiệm về tính trung thực, chính xác của nội dung luận văn này.

TP. Hồ Chí Minh, tháng ***[CẦN ĐIỀN:]*** *năm* ***[CẦN ĐIỀN:]***

Sinh viên thực hiện

[CẦN ĐIỀN: Họ và tên]

Tóm Tắt

Bối cảnh: Transcript do nhận dạng tiếng nói tự động tạo ra trong họp trực tuyến thường ngắn, nhiễu và có hiện tượng chuyển mã ngôn ngữ. Phân tích sắc thái cảm xúc trên nguồn dữ liệu này có thể hỗ trợ quan sát chất lượng trao đổi, nhưng không phải là cơ sở để suy luận hay chẩn đoán sức khỏe tâm thần của một cá nhân.

Mục tiêu: Khóa luận xây dựng và kiểm chứng một ứng dụng MVP xử lý transcript tiếng Việt trong họp trực tuyến theo hướng có thể truy vết bằng chứng, bảo vệ quyền riêng tư và cập nhật gần thời gian thực. Ứng dụng được thiết kế để nhận một bộ phân loại sentiment có thể thay thế; đầu ra dự kiến gồm ba lớp *positive*, *neutral*, *negative*; lớp *unknown* được giữ lại khi dữ liệu hoặc bộ phân loại không đủ điều kiện.

Phương pháp: Hệ thống tiếp nhận transcript từ Vexa, lưu phiên bản segment trong backend sở hữu dữ liệu, áp dụng quality gate và language router, ghép các segment cùng người nói thành analysis unit, sau đó gọi adapter phân loại và tổng hợp theo cửa sổ thời gian. Kiến trúc dùng FastAPI, PostgreSQL, Valkey Stream, worker tách riêng và SSE. PhoBERT và custom model là lựa chọn tích hợp về mặt lý thuyết [1]; runtime đã kiểm chứng của MVP là `baseline-rules-v2` dùng để kiểm tra luồng xử lý, không phải kết quả fine-tune hay đánh giá chất lượng mô hình.

Kết quả: Luận văn chuẩn bị biểu mẫu kiểm chứng pipeline và bảng so sánh nhiều model để điền sau quá trình build, huấn luyện lại và chạy thực nghiệm. Tại thời điểm biên soạn, không công bố giá trị định lượng về transcript, coverage, chất lượng sentiment, WER ASR, độ trễ hay năng lực sàng lọc trầm cảm. Dashboard được thiết kế để hiển thị KPI, timeline, evidence và cập nhật SSE khi có dữ liệu chạy thực tế.

Kết luận: MVP tạo nền tảng có thể kiểm toán để tích hợp bộ phân loại sentiment vào transcript họp. Huấn luyện lại, benchmark, calibration và đánh giá trải nghiệm là hướng tối ưu hóa tiếp theo, không phải đóng góp thực nghiệm của khóa luận này.

Từ khóa: *Phân tích sentiment, PhoBERT, transcript tiếng Việt, họp trực tuyến, event-driven, evidence, quyền riêng tư.*

Abstract

Background: Automatic-speech-recognition transcripts from online meetings are often short, noisy, and code-switched. Sentiment analysis can assist the observation of conversational content, but it must not be used to infer or diagnose an individual's mental health.

Objective: This thesis builds and verifies an evidence-traceable, privacy-aware, near-real-time application MVP for Vietnamese meeting transcripts. The application is designed to accept a replaceable sentiment classifier. The intended task comprises *positive*, *neutral*, and *negative* classes; *unknown* is retained when data or classifier conditions are insufficient.

Methods: The system ingests Vexa transcripts, versions segments in an application-owned backend, applies a quality gate and language router, constructs speaker-bounded analysis units, invokes a classifier adapter, and publishes windowed aggregates. FastAPI, PostgreSQL, Valkey Stream, a separate worker, and SSE form the implementation. PhoBERT and custom models are theoretical integration options [1]; the verified MVP runtime is `baseline-rules-v2`, which exercises the processing flow rather than claiming a fine-tuned-model evaluation.

Results: The thesis provides pipeline-verification and multi-model-comparison templates to be populated after build, retraining, and experimental runs. At the time of writing, no quantitative values for transcript volume, coverage, sentiment quality, ASR WER, or latency are reported. The dashboard is designed to display KPIs, a timeline, evidence, and SSE updates when actual run data are available.

Keywords: *Sentiment analysis, PhoBERT, Vietnamese transcript, online meeting, event-driven architecture, evidence, privacy.*

Mục lục

Lời cam đoan i

Tóm tắt ii

Abstract iii

Danh mục từ viết tắt ix

1 Giới Thiệu 1

1.1 Bối cảnh và tính cấp thiết của nghiên cứu . . . 1

1.2 Vấn đề nghiên cứu . . . 1

1.3 Mục tiêu, câu hỏi và phương pháp nghiên cứu . . . 2

1.4 Đối tượng, phạm vi và giới hạn . . . 3

1.5 Đóng góp của khóa luận . . . 4

1.6 Cấu trúc khóa luận . . . 4

2 Tổng Quan Nghiên Cứu và Cơ Sở Lý Thuyết 5

2.1 Sentiment analysis trong bối cảnh transcript họp . . . 5

2.1.1 Khái niệm và đơn vị phân tích . . . 5

2.1.2 Phân biệt các lớp bài toán . . . 5

2.2 Transcript ASR và kiểm soát chất lượng dữ liệu . . . 6

2.2.1 Nguồn sai lệch của transcript . . . 6

2.2.2 Quality gate, language routing và coverage . . . 6

2.3 Nền tảng mô hình ngôn ngữ và custom model . . . 7

2.3.1 Các lựa chọn mô hình cho tiếng Việt . . . 7

2.3.2 Hợp đồng adapter và vòng đời custom model . . . 7

2.4 Cơ sở thiết kế hệ thống có thể truy vết . . . 8

2.4.1 Hợp đồng dữ liệu, revision và evidence . . . 8

2.4.2 Luồng sự kiện và cập nhật gần thời gian thực . . . 8

2.4.3 Mở rộng đa nền tảng . . . 8

2.5 Nguyên tắc nghiên cứu và triển khai có trách nhiệm . . . 9

3 Dữ Liệu và Pipeline Phân Tích 10

3.1 Nguồn dữ liệu, đơn vị xử lý và quản trị dữ liệu . . . 10
3.1.1 Nguồn dữ liệu trong MVP . . . 10
3.1.2 Khái niệm và hợp đồng dữ liệu . . . 10
3.2 Sơ đồ pipeline và quá trình vận hành . . . 11
3.3 Các giai đoạn xử lý dữ liệu . . . 12
3.3.1 Ingest, chuẩn hóa và versioning . . . 12
3.3.2 Quality gate, language routing và xây analysis unit . . . 12
3.3.3 Suy luận, persistence và tổng hợp . . . 12
3.4 Tính đúng đắn vận hành và khả năng phục hồi . . . 13
3.5 Độ trễ, chỉ số và giới hạn đo lường . . . 13

4 Thiết Kế, Tích Hợp và Vòng Đời Mô Hình **14**
4.1 Trạng thái mô hình trong MVP . . . 14
4.2 Nguyên tắc thiết kế model cho hệ thống . . . 14
4.2.1 Hợp đồng adapter . . . 14
4.2.2 Các phương án model . . . 15
4.3 Thiết kế dữ liệu và protocol huấn luyện lại . . . 15
4.3.1 Dataset và gán nhãn . . . 15
4.3.2 Thiết kế experiment và artifact . . . 16
4.4 Đánh giá, calibration và điều kiện promote . . . 16
4.5 Retraining và giám sát sau triển khai . . . 16
4.6 Kiến trúc MLflow đề xuất . . . 17
4.7 Giới hạn thực nghiệm và phạm vi tuyên bố . . . 17

5 Kiến Trúc Backend và Tích Hợp Hệ Thống **18**
5.1 Yêu cầu thiết kế và nguyên tắc kiến trúc . . . 18
5.2 Kiến trúc logic và luồng tích hợp . . . 19
5.3 Mô hình dữ liệu backend và tính nhất quán . . . 19
5.3.1 Persistence do ứng dụng sở hữu . . . 19
5.3.2 Outbox, queue và khôi phục . . . 20
5.4 Giao tiếp bất đồng bộ và delivery cho dashboard . . . 20
5.5 Bảo mật tại các boundary . . . 20
5.5.1 Xác thực webhook và API key . . . 20
5.5.2 Identity ứng dụng và kiểm soát truy cập . . . 20
5.6 Khả năng vận hành và hướng tích hợp mở rộng . . . 21

6 Giao Diện Người Dùng và Dashboard **22**
6.1 Mục tiêu thiết kế và ranh giới sử dụng . . . 22
6.2 Thiết kế không gian thông tin và mô hình giao diện . . . 23
6.3 Kiến trúc frontend và tích hợp backend . . . 24

6.4 Mô hình trạng thái và quá trình vận hành giao diện . . . 24
6.5 Các thành phần trình bày và diễn giải output . . . 25
6.5.1 Tổng quan, timeline và speaker . . . 25
6.5.2 Evidence và trạng thái không chắc chắn . . . 25
6.6 Accessibility, privacy và an toàn giao diện . . . 25
6.7 Đánh giá UX và hướng tích hợp khi vận hành . . . 26

7 Hạ Tầng và Vận Hành **27**
7.1 Nguyên tắc thiết kế hạ tầng . . . 27
7.2 Topology Docker Compose đã kiểm chứng . . . 27
7.2.1 Hai miền triển khai và ranh giới mạng . . . 27
7.2.2 Dịch vụ, volume và health check . . . 28
7.3 Cấu hình, secret và network . . . 28
7.3.1 Một nguồn cấu hình runtime . . . 28
7.3.2 Port exposure và giao tiếp giữa service . . . 29
7.4 Quy trình build, khởi động và kiểm chứng . . . 29
7.5 Quan sát, sao lưu và khả năng khôi phục . . . 29
7.6 Kiến trúc vận hành mở rộng và giới hạn . . . 30

8 Kiểm Chứng Ứng Dụng và Thiết Kế Thực Nghiệm **31**
8.1 Câu hỏi kiểm chứng và nguyên tắc báo cáo . . . 31
8.2 Kiểm thử mức đơn vị và hợp đồng . . . 32
8.3 Kiểm chứng integration và vận hành . . . 32
8.3.1 Smoke pipeline tổng hợp . . . 32
8.3.2 Readiness, dashboard và contract nguồn . . . 33
8.4 Protocol đánh giá model sentiment . . . 33
8.4.1 Dữ liệu, nhãn và split . . . 33
8.4.2 Metric, uncertainty và so sánh model . . . 33
8.5 Đánh giá ASR, hệ thống và trải nghiệm người dùng . . . 34
8.5.1 ASR và transcript . . . 34
8.5.2 Hiệu năng và resilience hệ thống . . . 34
8.5.3 UX và cách diễn giải . . . 34
8.6 Biên bản bằng chứng cho một lần kiểm chứng integration . . . 35
8.7 Giới hạn thực nghiệm hiện tại . . . 35

9 Quyền Riêng Tư, Đạo Đức và An Toàn **37**
9.1 Mở đầu và phạm vi đánh giá . . . 37
9.2 Mô hình rủi ro và nguyên tắc thiết kế . . . 37
9.3 Consent, quản trị dữ liệu và quyền của chủ thể dữ liệu . . . 38
9.4 Kiểm soát dữ liệu theo vòng đời hệ thống . . . 38

9.5 Công bằng, độ tin cậy và diễn giải kết quả . . . 39
9.6 Ranh giới với an toàn và sức khỏe tâm thần . . . 39
9.7 Trạng thái hiện tại và điều kiện trước triển khai thực tế . . . 39

10 Kết Luận và Hướng Phát Triển **41**
10.1 Tổng kết nghiên cứu . . . 41
10.2 Bằng chứng và phạm vi kết luận . . . 41
10.3 Ý nghĩa đạo đức và vận hành . . . 42
10.4 Hạn chế . . . 42
10.5 Hướng phát triển có thể kiểm chứng . . . 42
10.6 Lời kết . . . 43

Tài liệu tham khảo **44**

Danh sách bảng

2.1 Ranh giới giữa các lớp bài toán liên quan đến transcript . . . 6

3.1 Các thực thể và trường truy vết cốt lõi của pipeline . . . 11

4.1 Các phương án model cho sentiment transcript tiếng Việt . . . 15

5.1 Yêu cầu kiến trúc backend và cơ chế hiện thực trong MVP . . . 18

6.1 Yêu cầu thiết kế dashboard và hiện thực trong MVP . . . 22

7.1 Các dịch vụ của Meeting Intelligence Compose . . . 28

8.1 Ma trận kiểm chứng và trạng thái bằng chứng của MVP . . . 32
8.2 Mẫu báo cáo so sánh model sentiment (chỉ điền sau run có thể tái lập) . . . 34
8.3 Nội dung cần ghi trong evidence bundle của một run integration . . . 35

Danh sách hình vẽ

3.1 Pipeline dữ liệu và vận hành của MVP. Nét đứt: hướng mở rộng chưa được kiểm chứng. . . . 11

4.1 Kiến trúc MLOps/MLflow đề xuất. Nét đứt: thành phần chưa được triển khai trong MVP. . . . 17

5.1 Kiến trúc logic backend của MVP. Nét đứt: hướng mở rộng chưa được hiện thực/kiểm chứng. . 19

6.1 Mô hình giao diện dashboard của MVP. Bố cục được xây dựng từ cấu trúc HTML/CSS/JavaScript hiện có và một ảnh vận hành đại diện; các giá trị trong hình là placeholder để tránh công bố dữ liệu cuộc họp. 23

6.2 Kiến trúc frontend và các boundary tích hợp. Nét đứt: hạng mục chưa được triển khai/kiểm chứng. 24

7.1 Topology triển khai cục bộ. Nét đứt biểu thị kết nối giữa hai Compose project; các domain dữ liệu được backup riêng. 28

Danh Mục Từ Viết Tắt

Viết tắt	Nghĩa đầy đủ
AI	Artificial Intelligence – Trí tuệ nhân tạo
API	Application Programming Interface
ASR	Automatic Speech Recognition – Nhận dạng giọng nói tự động
AUC	Area Under the Curve – Diện tích dưới đường cong ROC
BERT	Bidirectional Encoder Representations from Transformers
BIA	Business Impact Analysis – Phân tích tác động nghiệp vụ
BPE	Byte-Pair Encoding
CER	Character Error Rate – Tỷ lệ lỗi ký tự
CI/CD	Continuous Integration / Continuous Delivery (hoặc Deployment) – Tích hợp liên tục / phân phối (hoặc triển khai) liên tục
DPIA	Data Protection Impact Assessment – Đánh giá tác động bảo vệ dữ liệu
F1	F1 score – Trung bình điều hòa của precision và recall
GDPR	General Data Protection Regulation – Quy định bảo vệ dữ liệu chung của EU
HMAC	Hash-based Message Authentication Code – Mã xác thực thông điệp dựa trên hàm băm
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
JWT	JSON Web Token
KPI	Key Performance Indicator – Chỉ số hiệu quả chính
ML	Machine Learning – Học máy
MLM	Masked Language Modeling
MVP	Minimum Viable Product – Sản phẩm khả dụng tối thiểu
NFC	Normalization Form C – Dạng chuẩn hóa Unicode C
NLP	Natural Language Processing – Xử lý ngôn ngữ tự nhiên
OAuth	Open Authorization
OIDC	OpenID Connect

Viết tắt	Nghĩa đầy đủ
ONNX	Open Neural Network Exchange
RBAC	Role-Based Access Control – Kiểm soát truy cập dựa trên vai trò
REST	Representational State Transfer
RPO	Recovery Point Objective – Mục tiêu điểm khôi phục dữ liệu
RTO	Recovery Time Objective – Mục tiêu thời gian khôi phục
SLA	Service Level Agreement – Thỏa thuận mức dịch vụ
SLO	Service Level Objective – Mục tiêu mức dịch vụ
SSO	Single Sign-On – Đăng nhập một lần
SSE	Server-Sent Events
TLS	Transport Layer Security
UI	User Interface – Giao diện người dùng
URL	Uniform Resource Locator – Địa chỉ tài nguyên trên Web
UX	User Experience – Trải nghiệm người dùng
UUID	Universally Unique Identifier – Định danh duy nhất toàn cục
WCAG	Web Content Accessibility Guidelines – Hướng dẫn về khả năng truy cập nội dung Web
WER	Word Error Rate – Tỷ lệ lỗi từ

Chương 1

Giới Thiệu

1.1 Bối cảnh và tính cấp thiết của nghiên cứu

Sự phổ biến của họp trực tuyến làm cho một lượng lớn trao đổi công việc và học tập được chuyển thành dữ liệu âm thanh. Khi kết hợp với nhận dạng tiếng nói tự động (Automatic Speech Recognition – ASR), các trao đổi này có thể được biểu diễn dưới dạng transcript để tìm kiếm, rà soát và phân tích. Các mô hình ASR hiện đại đã cho thấy khả năng hoạt động trên nhiều ngôn ngữ và điều kiện âm thanh nhờ huấn luyện quy mô lớn [2]. Tuy nhiên, transcript sinh tự động không phải bản ghi chép hoàn hảo: tiếng ồn nền, chồng lấn lời nói, lỗi phân tách người nói, phát ngôn ngắn và hiện tượng chuyển mã Việt–Anh đều có thể làm thay đổi hoặc làm thiếu ngữ cảnh của văn bản đầu vào.

Trong bối cảnh đó, phân tích sắc thái cảm xúc (sentiment analysis) có thể hỗ trợ người dùng quan sát xu hướng biểu đạt của *nội dung trao đổi*, chẳng hạn tỷ trọng phát ngôn tích cực, trung tính hoặc tiêu cực theo thời gian. Với tiếng Việt, các mô hình ngôn ngữ tiền huấn luyện như PhoBERT là một lựa chọn đáng chú ý cho các tác vụ xử lý ngôn ngữ tự nhiên [1]. Dù vậy, năng lực của mô hình ngôn ngữ không loại bỏ các vấn đề thuộc hệ thống: dữ liệu vào cần được kiểm soát chất lượng, kết quả phải liên kết được với segment nguồn và thay đổi của transcript không được làm mất lịch sử phân tích.

Vì thế, giá trị của một ứng dụng trong miền này không chỉ nằm ở việc gọi một bộ phân loại. Hệ thống cần phân tách rõ dữ liệu thô, dữ liệu đủ điều kiện để suy luận và kết quả tổng hợp; đồng thời lưu phiên bản transcript, phiên bản mô hình, confidence, coverage và evidence để người dùng có thể kiểm tra ngữ cảnh của mỗi kết quả. Cách tiếp cận này phù hợp với nguyên tắc bảo vệ dữ liệu ngay từ thiết kế (privacy by design) [3] và nguyên tắc tối thiểu hóa dữ liệu trong GDPR [4].

Một ranh giới khoa học và đạo đức đặc biệt quan trọng là không đồng nhất sentiment với trạng thái sức khỏe tâm thần. Nhãn tích cực, trung tính hoặc tiêu cực của một analysis unit chỉ mô tả sắc thái của văn bản trong ngữ cảnh hẹp; nó không phải bằng chứng về đặc điểm, năng lực hay bệnh lý của người nói. Trầm cảm là tình trạng cần được đánh giá theo tiêu chí lâm sàng; PHQ-9 là công cụ tự báo cáo trong một khoảng thời gian xác định [5], còn chẩn đoán đòi hỏi đánh giá chuyên môn theo các tiêu chuẩn như DSM-5 [6]. Do đó, khóa luận không phát hiện hay sàng lọc trầm cảm, không xếp hạng cá nhân và không hỗ trợ quyết định về học tập, tuyển dụng hoặc việc làm.

1.2 Vấn đề nghiên cứu

Từ bối cảnh trên, bài toán được đặt ra là: làm thế nào xây dựng một MVP phân tích sentiment từ transcript tiếng Việt trong họp trực tuyến mà vẫn bảo toàn khả năng truy vết, thay thế mô hình và kiểm soát cách diễn giải

đầu ra? Bài toán được cụ thể hóa thành bốn vấn đề thiết kế và hiện thực:

(i) lọc các segment rỗng, quá ngắn, chưa hoàn tất, bị thu hồi hoặc có mẫu hallucination đã biết; lưu lại lý do loại thay vì âm thầm bỏ dữ liệu;

(ii) thiết kế hợp đồng adapter để bộ phân loại có thể được thay thế mà không làm thay đổi API, persistence và dashboard;

(iii) bảo toàn transcript gốc, revision và vòng đời phân tích trong kiến trúc cập nhật gần thời gian thực;

(iv) trình bày evidence, confidence và coverage để đầu ra được hiểu là hỗ trợ quan sát nội dung, không phải kết luận về con người.

Các vấn đề này gắn trực tiếp với kiến trúc của dự án. Ở luồng đã kiểm chứng, Google Meet là nguồn cuộc họp, Vexa cung cấp bot và transcript, còn ứng dụng sở hữu bản sao transcript, phiên bản phân tích và dashboard:

`Google Meet → Vexa → FastAPI/PostgreSQL → Valkey Stream → worker → SSE Dashboard.`

PostgreSQL là nguồn dữ liệu chuẩn; Valkey Stream chỉ phục vụ hàng đợi và fan-out, không thay thế dữ liệu bền vững. Worker tách khỏi FastAPI để suy luận không chặn API, còn dashboard chỉ giao tiếp với backend nên không tiếp xúc với khóa API của nhà cung cấp transcript. Cách phân tách này đồng thời tạo điều kiện kiểm toán lại chuỗi xử lý.

Kiến trúc được định hướng theo mô hình đa nền tảng. Bên ngoài lõi ứng dụng, mỗi nền tảng họp—Google Meet, Microsoft Teams, Zoom hoặc Jitsi Meet—cần một *platform adapter* chịu trách nhiệm xác thực URL/lịch họp, khởi tạo bot hoặc kết nối thu âm theo quyền được cấp, và chuẩn hóa transcript về cùng hợp đồng gồm định danh cuộc họp, speaker, timestamp, text, trạng thái `is_final` và revision. Sau bước chuẩn hóa, mọi dữ liệu đi qua cùng quality gate, worker, persistence và dashboard; vì vậy việc thêm platform không làm thay đổi ngữ nghĩa của dữ liệu phân tích. Trong phiên bản hiện tại, chỉ adapter Google Meet kết hợp Vexa được hiện thực và kiểm chứng. Adapter cho Teams, Zoom và Jitsi Meet là thiết kế mở rộng, chỉ được xem là đã hỗ trợ sau khi có contract nguồn, cơ chế consent, kiểm thử tích hợp và đánh giá riêng.

1.3 Mục tiêu, câu hỏi và phương pháp nghiên cứu

Mục tiêu tổng quát là xây dựng và kiểm chứng ở mức tích hợp một ứng dụng có khả năng tiếp nhận transcript tiếng Việt trong họp trực tuyến, áp dụng bộ phân loại sentiment có thể thay thế, và cung cấp kết quả có bằng chứng truy vết cùng các cơ chế bảo vệ quyền riêng tư. Khóa luận không đặt mục tiêu huấn luyện lại mô hình hoặc chứng minh chất lượng dự báo của một mô hình học máy cụ thể.

Để đạt mục tiêu này, nghiên cứu sử dụng hướng thiết kế–hiện thực–kiểm chứng artefact phần mềm: xác định yêu cầu và ràng buộc; thiết kế hợp đồng dữ liệu và kiến trúc; hiện thực MVP; sau đó kiểm chứng các luồng ingest, xử lý, lưu vết và hiển thị. Hướng tiếp cận này tương thích với quan điểm nghiên cứu khoa học thiết kế trong hệ thống thông tin, nơi artefact được xây dựng và đánh giá để giải quyết một lớp vấn đề xác định [7]. Đánh

giá trong khóa luận là kiểm chứng kỹ thuật của MVP, không phải đánh giá lâm sàng hay benchmark hiệu năng mô hình.

Câu hỏi nghiên cứu:

1. Pipeline có loại, lưu vết lý do và bảo toàn revision của transcript không đủ điều kiện trước khi gọi bộ phân loại hay không?
2. Hợp đồng adapter có cho phép thay thế bộ phân loại hiện tại bằng PhoBERT hoặc model artifact phù hợp mà không thay đổi API, persistence và dashboard hay không?
3. Kiến trúc event-driven có hoàn tất ingest, analysis lifecycle, persistence và cập nhật dashboard từ final segment hay không, đồng thời duy trì một hợp đồng có thể mở rộng sang platform adapter khác?
4. Evidence, confidence và coverage trên dashboard được biểu diễn như thế nào để tránh coi output là kết luận về con người?

1.4 Đối tượng, phạm vi và giới hạn

Đối tượng nghiên cứu là pipeline xử lý transcript và kiến trúc ứng dụng phân tích sentiment, không phải việc nhận dạng cảm xúc đa phương thức hay đánh giá sức khỏe tâm thần. Đơn vị xử lý là *analysis unit* được ghép từ các final segment liền kề của cùng người nói, có giới hạn về số segment và thời gian. Chỉ các unit tiếng Việt đạt quality gate mới được chuyển tới bộ phân loại; unit không phù hợp hoặc không được hỗ trợ giữ nhãn *unknown*.

MVP được kiểm chứng trong phạm vi Google Meet kết hợp Vexa ở môi trường cục bộ. Runtime hiện hành là rule baseline có tính xác định, phiên bản `baseline-rules-v2`, nhằm kiểm tra data contract, luồng hàng đợi, persistence và dashboard; đây không phải mô hình học máy và không được dùng để kết luận về chất lượng sentiment. Các notebook model sẵn có trong thư mục dự án chỉ phục vụ tham khảo hoặc khám phá. Đặc biệt, notebook phân loại trầm cảm nhị phân khác về task, nhãn và domain so với sentiment ba lớp của ứng dụng, nên không phải artifact được phục vụ trong MVP.

Hợp đồng `SentimentClassifier` mở đường cho *custom model* (mô hình tùy biến) theo từng tổ chức, miền nghiệp vụ hoặc tập dữ liệu đã được cho phép sử dụng. Một custom model chỉ có thể được đưa vào vận hành khi được đóng gói thành artifact có định danh version, khai báo rõ task/nhãn/ngôn ngữ, tương thích với hợp đồng input–output và có báo cáo đánh giá trên tập dữ liệu phù hợp. Mỗi prediction cần lưu model version, preprocessing version và evidence để có thể tái lập, so sánh hoặc rollback. Cơ chế này không đồng nghĩa với việc huấn luyện mô hình trên transcript của người dùng: việc thu thập, gán nhãn hoặc fine-tune dữ liệu theo từng khách hàng chỉ được thực hiện khi có cơ sở pháp lý, consent, chính sách lưu giữ dữ liệu và đánh giá rủi ro riêng.

Do chưa có corpus transcript họp được gán nhãn đúng task và protocol đánh giá tương ứng, khóa luận không công bố macro-F1, accuracy, calibration, WER ASR, độ trễ percentile hay kết quả đánh giá người dùng. PhoBERT, custom model, benchmark đa mô hình, đo WER, Kubernetes, high availability và triển khai thực tế trên Zoom, Teams hoặc Jitsi Meet là hướng phát triển tiếp theo. Khi có dữ liệu hợp pháp và đánh giá thực

nghiệm, các kết quả đó cần được báo cáo theo protocol tách dữ liệu theo cuộc họp, giữ test set độc lập và nêu rõ phiên bản model/dữ liệu.

1.5 Đóng góp của khóa luận

Trong giới hạn nêu trên, khóa luận đóng góp một artefact MVP và các quyết định thiết kế có thể tái sử dụng cho bài toán tương tự:

- một pipeline transcript có quality gate, định tuyến ngôn ngữ, ghép analysis unit theo người nói và lưu revision/evidence;
- một giao diện `SentimentClassifier` tách rời mô hình khỏi API, cơ sở dữ liệu và dashboard, tạo đường tích hợp có kiểm soát cho model artifact sau này;
- một kiến trúc event-driven với PostgreSQL là nguồn dữ liệu chuẩn, outbox/queue cho xử lý bất đồng bộ và SSE cho cập nhật dashboard;
- một định hướng adapter hóa nguồn transcript để Google Meet, Microsoft Teams, Zoom và Jitsi Meet có thể dùng chung lõi phân tích sau khi từng adapter được hiện thực và kiểm chứng;
- cách biểu diễn dashboard nhấn mạnh coverage, confidence, timeline và evidence, đi kèm ranh giới sử dụng nhằm giảm nguy cơ diễn giải quá mức.

1.6 Cấu trúc khóa luận

Chương 2 trình bày cơ sở lý thuyết và ranh giới giữa sentiment với sàng lọc lâm sàng. Chương 3 mô tả dữ liệu, hợp đồng và pipeline phân tích; Chương 4 nêu trạng thái mô hình, adapter và protocol đánh giá khi huấn luyện lại. Các Chương 5–7 trình bày hiện thực MVP ở backend, frontend và hạ tầng. Chương 8 báo cáo kiểm chứng tích hợp và giới hạn thực nghiệm; Chương 9 phân tích bảo vệ dữ liệu cùng các ràng buộc đạo đức; cuối cùng, Chương 10 tổng kết và đề xuất hướng phát triển.

Chương 2

Tổng Quan Nghiên Cứu và Cơ Sở Lý Thuyết

Chương này hệ thống hóa các khái niệm cần thiết để thiết kế ứng dụng phân tích sentiment từ transcript họp trực tuyến. Trọng tâm là sự phù hợp giữa bài toán phân tích ngôn ngữ, chất lượng dữ liệu ASR và các cơ chế hệ thống để kết quả có thể truy vết. Các nội dung về mô hình, đa nền tảng và vận hành được trình bày như cơ sở thiết kế; chúng không thay thế bằng chứng thực nghiệm được báo cáo ở Chương 8.

2.1 Sentiment analysis trong bối cảnh transcript họp

2.1.1 Khái niệm và đơn vị phân tích

Sentiment analysis, hay opinion mining, là nhóm kỹ thuật xác định khuynh hướng đánh giá hoặc sắc thái biểu đạt trong văn bản [8]. Trong khóa luận này, bài toán được giới hạn ở phân loại polarity ba lớp *positive*, *neutral* và *negative* trên một *analysis unit*; lớp *unknown* biểu thị dữ liệu không đủ điều kiện hoặc không thuộc miền/ngôn ngữ mà adapter hỗ trợ. Như vậy, *unknown* là một trạng thái an toàn của hệ thống, không phải một lớp sentiment.

Việc chọn analysis unit thay vì một token, một câu đơn lẻ hoặc toàn bộ cuộc họp là quyết định thiết kế nhằm cân bằng ngữ cảnh và độ trễ. Một unit được xây dựng từ tối đa ba final segment liền kề của cùng người nói, trong giới hạn thời gian của pipeline. Cách ghép này có thể giữ lại phủ định hoặc bổ ngữ nằm ở segment lân cận, đồng thời không gom nhiều lượt thoại khác người nói vào cùng một dự đoán. Tuy nhiên, unit vẫn không đủ để diễn giải ý định, mỉa mai hoặc quan hệ xã hội; các hiện tượng đó chỉ có thể được đánh giá khi có guideline và dữ liệu gán nhãn phù hợp.

2.1.2 Phân biệt các lớp bài toán

Sentiment, emotion, intent và đánh giá lâm sàng có mục tiêu, nhãn và yêu cầu dữ liệu khác nhau. Sự phân biệt này cần được duy trì từ khâu thu thập dữ liệu đến giao diện người dùng để tránh suy diễn vượt quá bằng chứng.

Bảng 2.1: Ranh giới giữa các lớp bài toán liên quan đến transcript

Bài toán	Đầu ra/dữ liệu cần có	Quan hệ với khóa luận
Sentiment	Polarity ở cấp analysis unit; guideline gán nhãn theo ngữ cảnh họp	Bài toán đích của kiến trúc ứng dụng
Emotion/intent	Nhiều nhãn cảm xúc hoặc mục đích phát ngôn; dữ liệu và guideline riêng	Hướng mở rộng, không suy ra từ nhãn sentiment
Tóm tắt/đánh giá cuộc họp	Ngữ cảnh nhiều lượt thoại, mục tiêu cuộc họp và tiêu chí đánh giá riêng	Không phải đầu ra của classifier hiện tại
Sàng lọc lâm sàng	Công cụ đã được thẩm định, khoảng thời gian xác định, consent và chuyên môn lâm sàng	Ngoài phạm vi

PHQ-9 là công cụ tự báo cáo trong một cửa sổ thời gian xác định [5]; chẩn đoán còn đòi hỏi đánh giá chuyên môn theo DSM-5 [6]. Vì vậy, không có phép ánh xạ trực tiếp từ sentiment của một transcript sang tiêu chí PHQ-9, DSM-5 hoặc trạng thái sức khỏe tâm thần của người nói. Hệ thống không được thiết kế để xếp hạng con người hoặc hỗ trợ quyết định học tập, tuyển dụng hay việc làm.

2.2 Transcript ASR và kiểm soát chất lượng dữ liệu

2.2.1 Nguồn sai lệch của transcript

ASR biến tín hiệu âm thanh thành chuỗi ký tự; các hệ thống hiện đại có thể hoạt động trên dữ liệu đa ngôn ngữ ở quy mô lớn [2]. Tuy nhiên, chất lượng transcript còn chịu ảnh hưởng của tiếng ồn, vọng âm, lời nói chồng lấn, tốc độ nói, tên riêng, từ viết tắt, chuyển mã ngôn ngữ và lỗi diarization. Với họp trực tuyến, một lỗi nhỏ cũng có thể đảo polarity, ví dụ làm mất từ phủ định hoặc gán phát ngôn sang sai speaker. Do đó, transcript phải được xem là dữ liệu quan sát có sai số, không phải ground truth ngôn ngữ.

Khi có audio gốc và bản chép chuẩn tương ứng, Word Error Rate (WER) có thể được xác định bởi:

$$WER = \frac{S + D + I}{N}, \tag{2.1}$$

trong đó S, D, I lần lượt là số lỗi thay thế, xóa và chèn; N là số từ trong bản chép chuẩn. Với tiếng Việt, việc định nghĩa token và quy tắc tách từ phải được cố định trước khi so sánh. WER/CER chỉ phản ánh sai khác so với ground truth đã chọn; chúng không trực tiếp đo đúng–sai của nhãn sentiment. Vì MVP chưa có audio và ground truth được thiết kế để benchmark, khóa luận không báo cáo WER/CER.

2.2.2 Quality gate, language routing và coverage

Trước suy luận, hệ thống chuẩn hóa Unicode NFC và khoảng trắng, đồng thời giữ nguyên dấu tiếng Việt. Quality gate đánh giá ít nhất các điều kiện: text rỗng, số từ quá ít, duration âm, segment chưa final, segment bị retracted và mẫu hallucination đã biết. Language router chỉ chuyển unit tiếng Việt đến adapter hiện hành; trường hợp khác hoặc không chắc chắn được giữ *unknown*. Segment bị loại vẫn lưu text đã chuẩn hóa và lý do loại để có thể audit, thay vì bị xóa hoặc ép thành *neutral*.

Để tránh diễn giải nhầm một tỷ lệ dự đoán cao là toàn bộ transcript đã được xử lý, hệ thống tách ba chỉ số. Với T là tổng số đơn vị được xem xét trước quality gate, A là số unit được chấp nhận và P là số unit có nhãn khác *unknown*, hệ thống định nghĩa:

$$acceptance_rate = \frac{A}{T}, \qquad prediction_coverage = \frac{P}{A}, \qquad effective_coverage = \frac{P}{T}. \tag{2.2}$$

Các tỷ lệ có mẫu số bằng 0 được báo là 0 kèm trạng thái "không có đơn vị phù hợp", không được suy diễn thành tỷ lệ sentiment. *Prediction coverage* thấp có thể do routing, ngưỡng confidence hoặc adapter; *acceptance rate* thấp phản ánh chất lượng đầu vào; còn *effective coverage* cho biết phần nội dung ban đầu thực sự nhận được dự đoán. Backend lưu cả ba chỉ số; dashboard hiện giữ *prediction coverage* để tương thích và cần bổ sung hai chỉ số còn lại trước khi công bố kết quả vận hành.

2.3 Nền tảng mô hình ngôn ngữ và custom model

2.3.1 Các lựa chọn mô hình cho tiếng Việt

Transformer dùng self-attention để biểu diễn quan hệ ngữ cảnh [9]:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V. \tag{2.3}$$

BERT là encoder Transformer được tiền huấn luyện bằng masked language modeling [10]; PhoBERT là mô hình tiền huấn luyện chuyên cho tiếng Việt [1]. Khi dùng PhoBERT, tokenizer, checkpoint và bước word segmentation phải tương thích; không áp dụng stemming hay loại stopword một cách mặc định vì có thể làm mất thông tin phủ định và ngữ cảnh.

TF–IDF kết hợp Logistic Regression, PhoBERT fine-tune và mBERT/XLM-R là các hướng so sánh hợp lý khi có dataset đúng task [11]. Không có mô hình nào mặc nhiên tốt nhất cho transcript họp: khác biệt về miền dữ liệu, cách ghi transcript, phân bố lớp và quy tắc gán nhãn có thể làm thay đổi kết quả. Vì vậy, PhoBERT trong khóa luận là một ứng viên tích hợp, không phải kết quả benchmark đã đạt.

2.3.2 Hợp đồng adapter và vòng đời custom model

Để tách phần hệ thống khỏi thuật toán, worker gọi giao diện `SentimentClassifier` thay vì phụ thuộc vào một framework mô hình cụ thể. Adapter nhận các analysis unit và trả về label, confidence, phân bố lớp (nếu có), polarity và model version. Nhờ hợp đồng này, baseline, mô hình PhoBERT hoặc custom model theo miền nghiệp vụ có thể được thay thế mà không sửa API, schema persistence hay dashboard.

Một custom model chỉ nên được đưa vào serving khi có artifact bất biến, version rõ ràng, mô tả task/nhãn/ngôn ngữ, preprocessing tương ứng, tập đánh giá độc lập và điều kiện rollback. Model cards đề xuất việc công bố mục đích sử dụng, giới hạn và kết quả đánh giá của mô hình [12]; tương tự, datasheets hỗ trợ ghi nhận nguồn gốc, cách thu thập và điều kiện sử dụng dữ liệu [13]. Các tài liệu này là cơ sở quản trị cho model và dataset, không phải bằng chứng rằng custom model đã được huấn luyện trong MVP.

Confidence do model xuất ra không tự động là xác suất đã calibration. Các mạng nơ-ron hiện đại có thể over-confident, do đó cần đo calibration trên tập validation/test độc lập trước khi diễn giải confidence như xác suất đúng [14]. Trong runtime `baseline-rules-v2`, confidence chỉ là điểm theo luật; nó không có ý nghĩa calibration và không phải thước đo về con người.

2.4 Cơ sở thiết kế hệ thống có thể truy vết

2.4.1 Hợp đồng dữ liệu, revision và evidence

Mỗi transcript segment được định danh bởi khóa nguồn trong một meeting và lưu text, speaker, timestamp, trạng thái final/retracted và revision. Khi text, speaker hoặc thời gian thay đổi, hệ thống tăng revision thay vì ghi đè mất lịch sử. Một analysis run gắn với source revision, model version và preprocessing version; prediction tiếp tục liên kết với unit và các segment nguồn. Chuỗi liên kết này tạo *evidence* cho phép người dùng đối chiếu một aggregate với văn bản, thời điểm và phiên bản xử lý đã sinh ra nó.

Tính truy vết đặc biệt quan trọng với transcript vì cùng một meeting có thể được nguồn cập nhật muộn hoặc sửa lại. Nếu chỉ lưu nhãn cuối cùng, hệ thống không thể trả lời nhãn đó được tạo từ bản transcript nào hoặc model nào. Ngược lại, revision và versioning cho phép supersede kết quả cũ, chạy lại phân tích khi có model mới và rollback mà không làm sai lệch lịch sử.

2.4.2 Luồng sự kiện và cập nhật gần thời gian thực

Kiến trúc MVP sử dụng PostgreSQL làm nguồn dữ liệu chuẩn. Sau khi transaction ghi segment hoặc kết quả phân tích được commit, outbox event bền vững được tạo để worker hoặc dashboard có thể tiêu thụ; Valkey Stream được dùng cho queue/fan-out, không phải nơi lưu dữ liệu chính. Khóa idempotency `(meeting_id, kind, source_revision, model_version)` hạn chế việc chạy lặp cùng một tác vụ phân tích khi có retry.

Worker xử lý bất đồng bộ để REST/SSE không bị chặn bởi quality gate hoặc inference. Sau khi aggregate cập nhật, backend phát sự kiện cho dashboard qua SSE. SSE là cơ chế HTTP một chiều từ server đến client, được chuẩn hóa để nhận luồng thông báo dạng event [15]; nó phù hợp với nhu cầu dashboard chủ yếu nhận cập nhật. Kiến trúc này chỉ cho phép tuyên bố *near-real-time* khi chưa có số liệu đo đầy đủ cho toàn bộ đường đi audio–transcript–dashboard.

2.4.3 Mở rộng đa nền tảng

Lõi phân tích cần độc lập với nền tảng họp. Theo thiết kế mở rộng, Google Meet, Microsoft Teams, Zoom và Jitsi Meet được đặt ở lớp *platform adapter*; adapter chuẩn hóa định danh meeting, speaker, timestamp, text, `is_final` và revision trước khi gửi vào pipeline chung. Mỗi adapter phải xử lý riêng cơ chế xác thực, quyền truy cập, consent, rate limit, định dạng webhook/polling và đặc tính transcript của nền tảng nguồn.

MVP hiện chỉ hiện thực và kiểm chứng luồng Google Meet kết hợp Vexa ở môi trường cục bộ. Teams, Zoom và Jitsi Meet không phải chức năng đã triển khai; chúng chỉ trở thành phạm vi triển khai khi từng adapter có

contract nguồn, kiểm thử tích hợp, kiểm soát quyền riêng tư và đánh giá độ tin cậy riêng. Cách diễn đạt này tránh suy diễn từ khả năng mở rộng của schema sang khả năng tương tác đã được chứng minh.

2.5 Nguyên tắc nghiên cứu và triển khai có trách nhiệm

Transcript họp có thể chứa dữ liệu cá nhân, bí mật kinh doanh hoặc thông tin nhạy cảm. Vì vậy, consent, mục đích sử dụng xác định, tối thiểu hóa dữ liệu, kiểm soát truy cập, retention/deletion và auditability là yêu cầu thiết kế. Privacy by design nhấn mạnh việc đưa bảo vệ riêng tư vào vòng đời hệ thống [3]; GDPR cũng đặt ra các nguyên tắc xử lý dữ liệu cá nhân như giới hạn mục đích và tối thiểu hóa dữ liệu [4].

Ứng dụng được thiết kế để hỗ trợ các nguyên tắc trên, không tuyên bố tuân thủ GDPR, AI Act hay pháp luật Việt Nam khi chưa có legal review và đánh giá tác động bảo vệ dữ liệu. EU AI Act có thể được dùng làm bối cảnh quản trị AI [16], nhưng việc áp dụng cho một deployment cụ thể cần phân tích theo mục đích sử dụng, vai trò các bên và khu vực pháp lý. Trong mọi trường hợp, output sentiment phải được hiển thị như tín hiệu về nội dung transcript, có evidence để rà soát và có human oversight; không được dùng như kết luận tự động về cá nhân.

Chương 3

Dữ Liệu và Pipeline Phân Tích

Chương này mô tả pipeline dữ liệu của MVP từ transcript nguồn đến dashboard. Mục tiêu của pipeline không chỉ là tạo nhãn sentiment, mà còn bảo toàn khả năng trả lời các câu hỏi: dữ liệu đến từ đâu, phiên bản nào đã được xử lý, vì sao một unit bị từ chối, mô hình nào tạo prediction và kết quả nào đang còn hiệu lực. Cách tổ chức này xem provenance là thuộc tính cần bảo toàn của dữ liệu, thay vì một thông tin bổ sung sau khi đã suy luận [17].

3.1 Nguồn dữ liệu, đơn vị xử lý và quản trị dữ liệu

3.1.1 Nguồn dữ liệu trong MVP

Nguồn dữ liệu vận hành của MVP là transcript họp Google Meet do Vexa cung cấp, được tiếp nhận bằng webhook có xác minh chữ ký hoặc bằng cơ chế reconcile REST dự phòng. Transcript chỉ được dùng để kiểm chứng luồng tích hợp khi có consent phù hợp. Backend không coi Vexa là kho dữ liệu ứng dụng: sau khi nhận, dữ liệu được lưu trong PostgreSQL do ứng dụng sở hữu để có thể version, phân tích lại và áp dụng chính sách retention/delete độc lập.

Các dataset và notebook có sẵn trong thư mục dự án không được dùng làm ground truth cho chất lượng của ứng dụng, vì domain, nhãn và đơn vị văn bản của chúng có thể khác transcript họp. Role-play hoặc transcript tổng hợp chỉ phù hợp với smoke test; chúng không thay thế dữ liệu đánh giá có gán nhãn. Nếu xây dựng custom model trong tương lai, dataset cần có license, mục đích sử dụng, cách thu thập, thành phần dữ liệu và giới hạn được ghi nhận theo tinh thần datasheets for datasets [13].

3.1.2 Khái niệm và hợp đồng dữ liệu

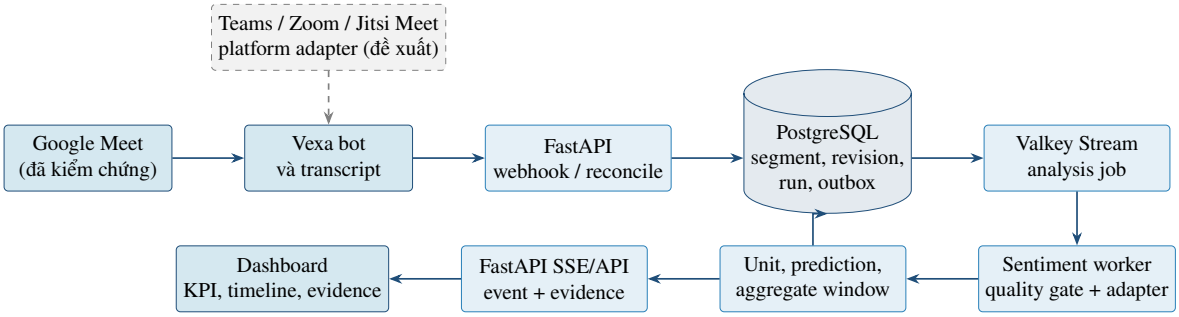
Pipeline dùng các khái niệm ở Bảng 3.1 để tách dữ liệu gốc khỏi dữ liệu suy luận và dữ liệu trình bày. Hợp đồng thống nhất này cũng là điểm nối cho platform adapter: Google Meet đã được kiểm chứng; Teams, Zoom và Jitsi Meet chỉ có thể tham gia pipeline sau khi adapter của từng nền tảng chuẩn hóa được các trường tương đương và được kiểm thử riêng.

Bảng 3.1: Các thực thể và trường truy vết cốt lõi của pipeline

Khái niệm	Định nghĩa trong hệ thống
Raw segment	Segment transcript từ nguồn, gồm định danh nguồn, speaker, text, timestamp, ngôn ngữ và trạng thái. Dữ liệu gốc được giữ trong PostgreSQL.
Final segment	Segment có `is_final=true` và không bị `retracted`; chỉ loại này được xét để tạo analysis unit.
Revision	Số phiên bản tăng khi text, speaker hoặc thời gian của segment thay đổi. Meeting có `transcript_revision` để gắn toàn bộ analysis run với ảnh chụp nguồn.
Analysis unit	Một đến ba final segment liền kề cùng speaker, giới hạn trong cửa sổ 45 giây; là đơn vị đầu vào cho classifier.
Prediction	Label, confidence, phân bố lớp (nếu adapter cung cấp), polarity, thời gian inference và `model_version`.
Evidence	Liên kết từ aggregate/prediction đến unit, segment ID, revision, text chuẩn hóa, timestamp, preprocessing version và model version.
Coverage	Tỷ lệ unit có prediction khác *unknown*; được hiển thị cùng kết quả tổng hợp.

3.2 Sơ đồ pipeline và quá trình vận hành

Hình 3.1 biểu diễn luồng dữ liệu đã hiện thực. Mũi tên liền là đường xử lý của MVP Google Meet–Vexa; khung nét đứt biểu thị lớp adapter mở rộng và không phải tính năng đã kiểm chứng. PostgreSQL giữ bản ghi bền vững và outbox; Valkey Stream chỉ vận chuyển job cho worker. Dashboard nhận event qua SSE từ backend và đọc dữ liệu phân tích qua API, không gọi Vexa trực tiếp.



Hình 3.1: Pipeline dữ liệu và vận hành của MVP. Nét đứt: hướng mở rộng chưa được kiểm chứng.

Quy trình vận hành bắt đầu khi người dùng yêu cầu tạo bot cho Google Meet. Vexa thu transcript; backend nhận webhook khi nguồn phát sự kiện hoặc reconcile lại transcript qua REST khi cần. Segment được upsert theo khóa `(meeting_id, vexa_segment_id)`. Nếu nội dung, speaker hoặc thời gian thay đổi, revision của segment và `transcript_revision` của meeting được tăng, đồng thời tạo outbox event có dedupe key. Khi meeting hoàn tất hoặc người dùng yêu cầu phân tích, backend tạo một `analysis_run` gắn với source revision, model version và preprocessing version rồi đưa job vào Valkey Stream.

Worker đọc job theo consumer group, tạo analysis unit, gọi classifier adapter, lưu prediction và aggregate theo cửa sổ một phút. Khi run thành công, worker tạo `analysis.sentiment.updated` trong outbox. Endpoint SSE đọc outbox theo meeting, phát event cùng keepalive cho dashboard; dashboard sau đó làm mới overview, timeline và evidence qua API. Nếu Valkey không sẵn sàng khi enqueue, analysis run vẫn được lưu ở trạng thái `queued`; worker có cơ chế quét các run đang chờ để phục hồi xử lý. Đây là lý do PostgreSQL, thay vì

queue, được chọn làm nguồn dữ liệu chuẩn.

3.3 Các giai đoạn xử lý dữ liệu

3.3.1 Ingest, chuẩn hóa và versioning

Tại ingest, pipeline giữ text gốc từ provider cùng metadata nguồn, không chỉnh sửa nội dung để "làm đẹp" transcript. Bản text chuẩn hóa chỉ dùng trong bước phân tích: Unicode NFC được chuẩn hóa và whitespace được rút gọn, trong khi dấu tiếng Việt được giữ lại. Việc tách raw data và normalized text giúp hệ thống vừa tái lập được xử lý vừa cho phép kiểm tra lại dữ liệu nguồn.

Versioning xử lý hai dạng thay đổi: thay đổi transcript của provider và thay đổi cách phân tích. Thay đổi thứ nhất tạo segment/meeting revision; thay đổi thứ hai được biểu diễn bởi `model_version` và `preprocessing_version`. Kết quả cũ được đánh dấu không còn hiện hành (`is_current=false`) thay vì bị ghi đè. Nhờ đó, cùng một cuộc họp có thể được phân tích lại mà không mất liên kết tới phiên bản dữ liệu và mô hình đã tạo ra mỗi prediction.

3.3.2 Quality gate, language routing và xây analysis unit

Quality gate từ chối segment rỗng, dưới ba token, có duration âm, chưa final, bị retracted hoặc khớp mẫu hallucination đã biết. Lý do từ chối được lưu trong `quality_reasons`. Với segment hợp lệ, language router ưu tiên dấu hiệu tiếng Việt trong text, sau đó mới tham chiếu ngôn ngữ do nguồn cung cấp. Runtime hiện tại chỉ xử lý `vi`; ngôn ngữ khác trả *unknown* thay vì bị ép vào một lớp sentiment.

Các segment accepted được ghép theo cùng speaker, tối đa ba segment và không vượt quá 45 giây tính từ segment đầu. Nếu speaker thay đổi, thời gian vượt ngưỡng hoặc gặp segment bị từ chối, unit đang mở được flush. Pipeline cũng loại duplicate gần nhau của cùng speaker để một câu bị nguồn phát lặp không làm méo aggregate. Các quy tắc này là quyết định tiền xử lý của hệ thống; chúng phải được ghi version và đánh giá lại nếu thay classifier hoặc thay nguồn transcript.

3.3.3 Suy luận, persistence và tổng hợp

Chỉ analysis unit được chấp nhận mới đi vào `SentimentClassifier`. Adapter hiện hành là `baseline-rules-v2`; mục đích là kiểm chứng data contract và vòng đời xử lý, không phải đo chất lượng sentiment. Với custom model sau này, adapter phải trả về đúng hợp đồng prediction, đồng thời model artifact, preprocessing, tập đánh giá và giới hạn sử dụng cần được version hóa và ghi nhận [12].

Sau inference, pipeline lưu prediction theo cặp duy nhất `(analysis_run_id, analysis_unit_id)`. Aggregate được tạo ở hai scope: toàn meeting và theo speaker; mỗi scope được bucket theo phút. Với unit i, polarity của adapter được xác định:

$$p_i = P_i(positive) - P_i(negative), \qquad score_{window} = \frac{1}{N}\sum_{i=1}^{N} p_i. \tag{3.1}$$

Chỉ prediction có nhãn *positive*, *neutral* hoặc *negative* tham gia aggregate. Nếu $N = 0$, score được lưu là `null`;

dashboard không nên diễn giải trường hợp này là trạng thái trung tính. Trong runtime `baseline-rules-v2`, confidence là điểm theo luật chưa calibration; vì vậy aggregate hiện dùng đếm nhãn/mean không trọng số, không nhân theo confidence hoặc token count. Polarity chỉ là heuristic trực quan của adapter, không phải metric nghiên cứu hay xác suất về người nói. Chỉ sau khi model có calibration evaluation độc lập mới xem xét weighting có biện minh [14].

3.4 Tính đúng đắn vận hành và khả năng phục hồi

Pipeline hướng tới xử lý ít nhất một lần ở lớp queue; vì vậy idempotency và persistence là điều kiện cần để retry không nhân bản kết quả. `analysis_runs` có ràng buộc duy nhất trên `(meeting_id, kind, source_revision, model_version)`; outbox cũng có dedupe key. Worker chỉ acknowledge message Valkey sau khi xử lý và commit thành công. Nếu xử lý lỗi, transaction được rollback, run được ghi lỗi và message chưa được acknowledge để có thể quan sát/khôi phục theo cơ chế vận hành.

Webhook được xác minh HMAC kèm timestamp để giảm nguy cơ giả mạo và replay; event ID được lưu trong `incoming_webhooks` để loại event trùng. REST reconcile vẫn cần thiết vì webhook có thể đến muộn hoặc thất bại. Các cơ chế này tăng khả năng phục hồi của pipeline, nhưng không thay thế yêu cầu giám sát queue lag, lỗi API, retry và backup/restore PostgreSQL trong môi trường production.

3.5 Độ trễ, chỉ số và giới hạn đo lường

Độ trễ end-to-end được tách thành:

$$T_{e2e} = T_{capture} + T_{ingest} + T_{queue} + T_{analysis} + T_{delivery}, \tag{3.2}$$

trong đó $T_{capture}$ là thời gian từ audio đến final segment; các thành phần sau lần lượt là ingest, chờ queue, xử lý và phát lên dashboard. Cách phân rã này giúp xác định nút thắt thay vì chỉ báo cáo một số latency tổng. Trong MVP, chưa có log đo đầy đủ từng thành phần và chưa có benchmark tải; do đó tuyên bố phù hợp là *near-real-time*, không phải một SLA định lượng.

Các chỉ số cần đo khi mở rộng gồm: số segment/unit, tỷ lệ bị từ chối theo lý do, coverage, phân bố lớp, tỷ lệ retry/failure, queue lag, P50/P95 của $T_{analysis}$ và T_{e2e}, cùng mức sử dụng tài nguyên. WER/CER chỉ báo cáo khi có audio và ground truth phù hợp; macro-F1, calibration và metric theo lớp chỉ báo cáo khi có dataset sentiment đúng task, chia tập theo meeting và test set độc lập. Không có số liệu nào trong nhóm này được suy ra từ smoke test hoặc rule baseline.

Chương 4

Thiết Kế, Tích Hợp và Vòng Đời Mô Hình

Chương này trình bày mô hình như một thành phần có vòng đời trong hệ thống, không chỉ là một checkpoint được gọi tại thời điểm suy luận. Trọng tâm gồm hợp đồng adapter, điều kiện lựa chọn model, thiết kế artifact, retraining và quản trị phiên bản. Các quy trình MLflow được trình bày như kiến trúc MLOps đề xuất; MVP hiện chưa cài đặt MLflow hoặc serving một model học máy đã huấn luyện.

4.1 Trạng thái mô hình trong MVP

MVP hiện triển khai giao diện `SentimentClassifier` và runtime `baseline-rules-v2`. Baseline là bộ luật có tính xác định, nhằm kiểm tra data contract, quality gate, queue, persistence, evidence và dashboard. Nó không phải mô hình học máy, confidence của nó không được calibration và không được dùng để kết luận về chất lượng sentiment.

Worker khởi tạo baseline từ cấu hình `MODEL_VERSION`; nếu cấu hình yêu cầu version khác nhưng adapter tương ứng không được cài đặt, worker dừng thay vì tự động rơi về baseline. Ràng buộc này tránh trường hợp dashboard ghi nhận một model version nhưng thực tế suy luận bằng model khác. Mỗi analysis run hiện lưu `source_revision`, `model_version` và `preprocessing_version`; đây là nền tảng cần thiết để tích hợp một artifact học máy có kiểm soát.

Các notebook và dữ liệu tồn tại trong dự án trước đó thuộc bài toán phân loại trầm cảm nhị phân trên văn bản ngoài domain hợp, với protocol chia dữ liệu khác. Chúng không phải model artifact được serving, không có bằng chứng phù hợp cho sentiment ba lớp và không được dùng làm kết quả thực nghiệm của khóa luận. Không thể chuyển trực tiếp accuracy của các notebook này thành chất lượng của dashboard sentiment.

4.2 Nguyên tắc thiết kế model cho hệ thống

4.2.1 Hợp đồng adapter

Thiết kế tách lõi hệ thống khỏi thư viện học máy bằng giao diện `SentimentClassifier`. Adapter nhận danh sách analysis unit đã qua quality gate và trả về, cho mỗi unit, nhãn, confidence, phân bố lớp, polarity score, inference latency và version model. Nhờ đó, API, schema persistence, outbox và dashboard không phụ thuộc trực tiếp vào PyTorch, scikit-learn hoặc một checkpoint cụ thể.

Một adapter được xem là tương thích khi thỏa bốn điều kiện: (i) chỉ nhận text và language được khai báo trong model card; (ii) trả về nhãn thuộc tập *positive*, *neutral*, *negative*, *unknown*; (iii) gắn mọi prediction với version bất biến của artifact và preprocessing; và (iv) trả lỗi tường minh khi artifact không tải được hoặc input ngoài phạm vi. Thiết kế này hạn chế nợ kỹ thuật do phụ thuộc dữ liệu, cấu hình và các ranh giới hệ thống bị xói

mòn—những rủi ro phổ biến của hệ thống ML vận hành lâu dài [18].

4.2.2 Các phương án model

Lựa chọn model phải dựa trên dữ liệu target, ngân sách độ trễ, khả năng tái lập và rủi ro vận hành, không chỉ dựa trên một metric đơn lẻ. Bảng 4.1 tóm tắt các lựa chọn cần được đánh giá trên cùng protocol.

Bảng 4.1: Các phương án model cho sentiment transcript tiếng Việt

Phương án	Vai trò	Ưu điểm	Điều kiện trước khi serving
TF–IDF + Logistic Regression	Baseline học máy	Nhẹ, CPU-friendly, dễ phân tích lỗi	Corpus đúng task, vocabulary và preprocessing được version hóa
PhoBERT fine-tune	Ứng viên chính cho tiếng Việt	Biểu diễn ngữ cảnh tiếng Việt tốt hơn mô hình bag-of-words	Tokenizer, word segmentation, checkpoint và max length tương thích [1]
mBERT/XLM-R	Comparator đa ngôn ngữ	Có thể khảo sát dữ liệu đa ngôn ngữ sau này	Benchmark riêng cho language routing và code-switching [11]
Custom model theo miền	Ứng viên theo tổ chức/miền nghiệp vụ	Có thể phù hợp thuật ngữ và phong cách giao tiếp đặc thù	Dữ liệu hợp pháp, model card, đánh giá độc lập và quy trình phê duyệt

ViHealthBERT chỉ phù hợp khi task, license, model card và domain dữ liệu tương thích [19]; nó không phải lựa chọn mặc định cho transcript họp. PhoBERT hay bất kỳ custom model nào chỉ được gọi là tốt hơn baseline sau khi có benchmark tái lập trên tập test đã khóa.

4.3 Thiết kế dữ liệu và protocol huấn luyện lại

4.3.1 Dataset và gán nhãn

Retraining đòi hỏi corpus sentiment đúng task, trong đó đơn vị gán nhãn phải nhất quán với analysis unit mà runtime sử dụng. Guideline cần định nghĩa ba lớp polarity, trường hợp thiếu ngữ cảnh, phủ định, mỉa mai, code-switching, trích dẫn lời người khác và segment không đủ điều kiện. Tối thiểu hai annotator độc lập, adjudication và chỉ số agreement như Cohen's kappa hoặc Krippendorff's alpha là protocol khuyến nghị.

Tập dữ liệu phải được chia theo **meeting**, không chia ngẫu nhiên theo segment, để tránh leakage từ speaker, chủ đề hoặc ngữ cảnh của cùng một cuộc họp vào cả train và test. Test set được khóa trước khi tuning; threshold, prompt (nếu có) và hyperparameter chỉ được chọn trên validation. Dataset manifest cần ghi nguồn, consent/license, tiêu chí lọc, số meeting/speaker/unit, phân bố lớp, hash phiên bản, ngày tạo và giới hạn sử dụng. Cách ghi nhận này phù hợp với mục tiêu minh bạch của datasheets for datasets [13].

4.3.2 Thiết kế experiment và artifact

Mỗi experiment cần có một định danh có thể tái lập, tối thiểu gồm: commit mã nguồn, dataset manifest/hash, split, preprocessing version, tokenizer, checkpoint gốc, hyperparameter, seed, môi trường chạy và hardware. Artifact được promote không chỉ là file trọng số; nó cần bao gồm tokenizer, label mapping, threshold, model signature, dependency lockfile, evaluation report và model card. Model cards hỗ trợ ghi nhận mục đích, nhóm người dùng dự kiến, dữ liệu đánh giá, metric, giới hạn và các trường hợp không nên sử dụng model [12].

Không dùng transcript vận hành để huấn luyện mặc định. Việc trích xuất, gán nhãn hoặc fine-tune dữ liệu của một tổ chức chỉ được thực hiện khi có mục đích đã xác định, cơ sở pháp lý/consent, chính sách retention/delete và kiểm soát truy cập phù hợp. Custom model phải được quản trị tách biệt theo tổ chức hoặc workspace để tránh trộn dữ liệu và artifact giữa các miền không được phép.

4.4 Đánh giá, calibration và điều kiện promote

Một ứng viên model cần được so sánh với baseline trên cùng quality gate, language routing, test set và hardware. Báo cáo tối thiểu gồm macro-F1; precision, recall, F1 theo lớp; confusion matrix; coverage; phân bố nhãn; P50/P95 inference; và error analysis theo các lát dữ liệu đã xác định trước. Accuracy đơn lẻ không đủ khi lớp neutral chiếm ưu thế hoặc chi phí lỗi khác nhau giữa các lớp.

Confidence chỉ có thể được diễn giải là xác suất đúng sau khi kiểm tra calibration trên dữ liệu độc lập. Nghiên cứu về calibration cho thấy các mạng nơ-ron hiện đại có thể over-confident [14]; vì vậy báo cáo phải tách score thô, phương pháp calibration (nếu có) và metric như Expected Calibration Error. Nếu không đạt tiêu chí định trước, model vẫn có thể được giữ ở trạng thái nghiên cứu nhưng không được promote vào serving.

Điều kiện promote cần được xác định trước experiment, ví dụ: không giảm macro-F1 quá ngưỡng cho phép, không suy giảm nghiêm trọng một lớp, coverage và latency đáp ứng ngân sách, kiểm tra privacy/license đạt yêu cầu, và có evidence để rollback. Các ngưỡng số cụ thể chỉ được điền sau khi đội dự án xác định SLO và chạy đo thực tế; khóa luận không đặt số liệu giả định.

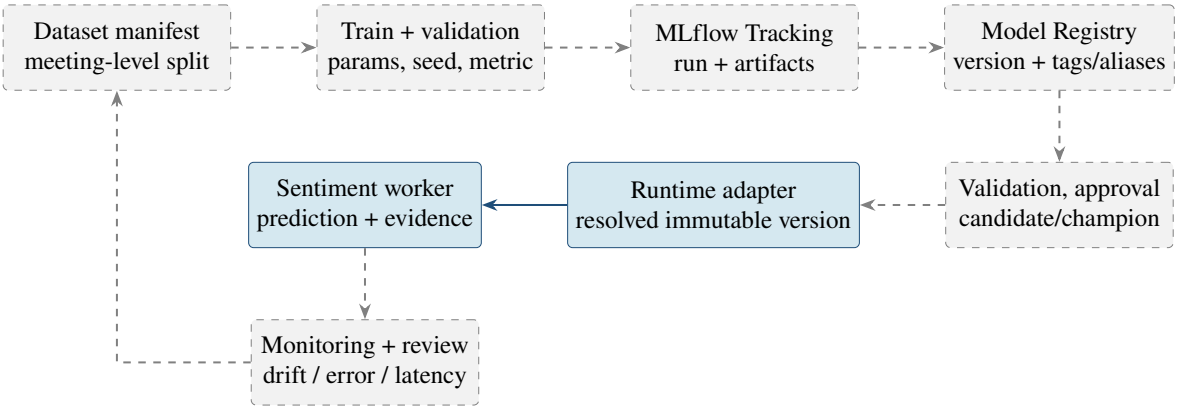
4.5 Retraining và giám sát sau triển khai

Retraining không nên kích hoạt chỉ vì có dữ liệu mới. Các tín hiệu có thể khởi tạo một vòng đánh giá gồm: thay đổi nguồn transcript hoặc language routing; thay đổi domain/thuật ngữ; tỷ lệ unit bị loại hoặc *unknown* tăng; drift phân bố token/nhãn; lỗi được người dùng đánh dấu; hoặc benchmark định kỳ cho thấy model suy giảm. Một trigger chỉ tạo yêu cầu đánh giá, không tự động đưa model mới vào production.

Vòng đời retraining đề xuất gồm: (1) phê duyệt phạm vi dữ liệu và cập nhật manifest; (2) tạo dataset version và meeting-level split; (3) chạy train/validation có tracking; (4) kiểm tra metric, calibration, latency và error analysis; (5) đăng ký artifact candidate; (6) shadow mode hoặc canary với logging có kiểm soát; (7) promote sau phê duyệt; và (8) giám sát/rollback. Shadow mode chạy candidate song song với champion nhưng không thay đầu ra dashboard; canary chỉ chuyển một phần traffic khi hệ thống đã có cơ chế định tuyến phù hợp. MVP hiện chưa hiện thực hai cơ chế này.

4.6 Kiến trúc MLflow đề xuất

MLflow cung cấp các khái niệm tracking experiment, lưu artifact và model registry cho vòng đời ML [20]. Theo tài liệu MLflow hiện hành, registry có thể quản lý version, tag và alias cho model; alias như `candidate` hoặc `champion` giúp tham chiếu phiên bản được chọn mà không dùng "latest" mơ hồ [21]. Hình 4.1 mô tả cách tích hợp đề xuất; toàn bộ các nút nét đứt là kiến trúc tương lai, không phải service đang chạy trong Docker Compose của MVP.



Hình 4.1: Kiến trúc MLOps/MLflow đề xuất. Nét đứt: thành phần chưa được triển khai trong MVP.

Để hiện thực kiến trúc này, cần bổ sung ít nhất bốn thay đổi có chủ đích. Thứ nhất, môi trường training dùng MLflow Tracking để log param, metric, dataset manifest và artifact; backend store của registry phải dùng cơ sở dữ liệu bền vững, còn artifact store cần phân quyền và backup. Thứ hai, registry đăng ký một version bất biến kèm model signature, model card và báo cáo đánh giá; alias chỉ phục vụ chọn candidate/champion, không thay cho bằng chứng version. Thứ ba, worker cần thêm adapter tải artifact từ `MODEL_URI` hoặc registry alias tại thời điểm khởi động, kiểm tra signature/preprocessing và lưu *resolved version* vào `analysis_run`. Thứ tư, cấu hình cần bổ sung URI MLflow, tên/alias model, timeout, credential và chính sách fail-closed; nếu registry hoặc artifact không sẵn sàng, worker không được âm thầm dùng baseline.

Khi promote, alias `champion` có thể được chuyển sang version đã phê duyệt, nhưng deployment phải resolve alias thành version bất biến, ghi version đó vào evidence và chỉ đổi runtime theo rollout có kiểm tra sức khỏe. Rollback là thao tác đưa deployment về artifact đã biết, thay vì huấn luyện lại khẩn cấp. Cách quản trị này giảm sự phụ thuộc ngầm giữa dữ liệu, code, artifact và cấu hình, đồng thời giữ khả năng tái lập của mỗi prediction.

4.7 Giới hạn thực nghiệm và phạm vi tuyên bố

Khóa luận chưa có artifact PhoBERT/custom model được serving, dataset sentiment transcript được gán nhãn, benchmark đa model, calibration run, drift monitor, MLflow server, model registry, shadow mode hoặc canary deployment. Vì vậy, không công bố macro-F1, AUC, accuracy, latency percentile, SLO hay hiệu quả retraining. Các nội dung trong chương là thiết kế và protocol để triển khai/đánh giá tiếp theo; bằng chứng đã chạy được giới hạn ở integration của `baseline-rules-v2` trong pipeline.

Chương 5

Kiến Trúc Backend và Tích Hợp Hệ Thống

Backend là lớp ranh giới giữa nền tảng transcript, dữ liệu ứng dụng, worker phân tích và dashboard. Mục tiêu thiết kế không phải là thay thế nhà cung cấp họp trực tuyến, mà là tiếp nhận dữ liệu có kiểm soát, duy trì bản ghi thuộc sở hữu ứng dụng, điều phối phân tích bất đồng bộ và trả kết quả có evidence. Chương này mô tả implementation MVP trên FastAPI, PostgreSQL, Valkey và Vexa; các thành phần production nâng cao được tách rõ khỏi phạm vi đã kiểm chứng.

5.1 Yêu cầu thiết kế và nguyên tắc kiến trúc

API HTTP cung cấp giao diện tài nguyên meeting, transcript và analytics cho dashboard. Phong cách REST nhấn mạnh interface thống nhất, triển khai độc lập các thành phần và khả năng đặt intermediary tại ranh giới hệ thống [22]; trong dự án, FastAPI là intermediary này giữa browser, Vexa và các store do ứng dụng quản lý. Tuy nhiên, việc dùng HTTP endpoint không tự động làm một hệ thống "RESTful" hay an toàn; các ràng buộc về ownership, idempotency, chữ ký webhook và dữ liệu bền vững vẫn phải được hiện thực riêng.

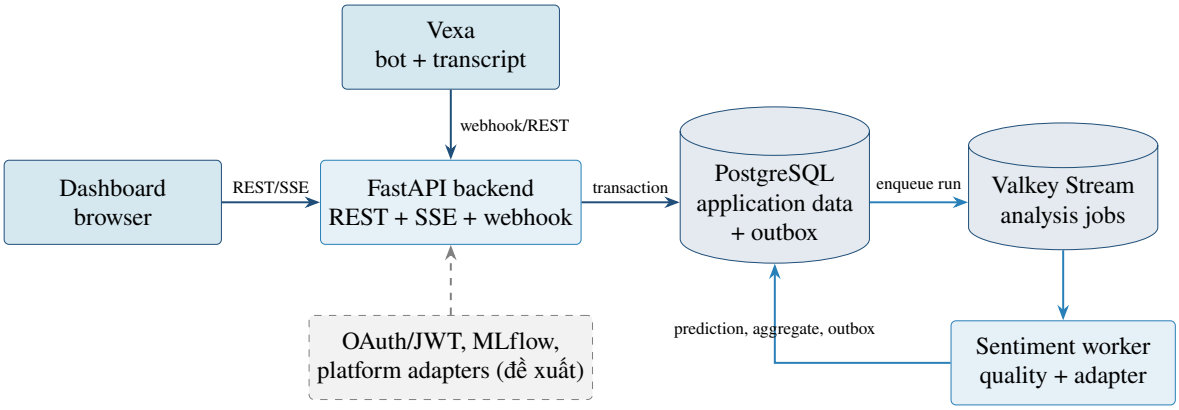
Các quyết định thiết kế chính được tóm tắt ở Bảng 5.1. Chúng hướng tới khả năng truy vết và khôi phục trước khi tối ưu hóa thông lượng hoặc mở rộng hạ tầng.

Bảng 5.1: Yêu cầu kiến trúc backend và cơ chế hiện thực trong MVP

Yêu cầu	Cơ chế hiện thực	Giới hạn hiện tại
Ownership và phân quyền dữ liệu	Meeting gắn `owner_id`; mọi API đọc/ghi meeting kiểm tra ownership	`X-App-User-Id` là identity development, chưa phải JWT/OAuth
Nguồn dữ liệu bền vững	PostgreSQL lưu meeting, segment, revision, run, unit, prediction, aggregate và outbox	Chưa có backup/restore drill hoặc HA được kiểm chứng
Không chặn API khi phân tích	Valkey Stream và worker tách riêng xử lý analysis run	Chưa có load test, autoscaling hay dynamic batching
Idempotency và retry	Unique constraint cho segment, webhook, analysis run và outbox dedupe key	Semantics delivery cần được đánh giá thêm ở quy mô production
Truy vết kết quả	Source revision, preprocessing/model version, evidence và `is_current`	Chưa có model registry/MLflow runtime
Delivery gần thời gian thực	SSE phát event outbox; dashboard refresh API theo meeting	SSE hiện poll outbox mỗi giây; chưa có WebSocket/direct push

5.2 Kiến trúc logic và luồng tích hợp

Hình 5.1 cho thấy các boundary chính. Browser không gọi Vexa và không chứa Vexa API key. Vexa chỉ giao tiếp với backend qua client server-side, webhook và REST reconcile. PostgreSQL là source of truth; Valkey Stream chỉ nhận định danh analysis run để worker xử lý. Sau khi worker commit prediction/aggregate, backend cung cấp SSE và API analytics cho dashboard.



Hình 5.1: Kiến trúc logic backend của MVP. Nét đứt: hướng mở rộng chưa được hiện thực/kiểm chứng.

Luồng tạo meeting bắt đầu từ `POST /v1/meetings`. Backend chỉ chấp nhận URL Google Meet đúng định dạng trong MVP, kiểm tra một meeting đang hoạt động cùng owner để request retry không spawn bot trùng, sau đó gọi Vexa bằng API key server-side. Các endpoint `/sync`, `/transcript`, `/analysis` và `/analytics/*` chỉ thao tác trên meeting thuộc owner đang gọi. Việc giới hạn API hiện tại ở Google Meet là chủ đích: Teams, Zoom và Jitsi Meet cần parser, contract transcript, cơ chế cấp quyền và kiểm thử adapter riêng trước khi được công bố là tích hợp.

5.3 Mô hình dữ liệu backend và tính nhất quán

5.3.1 Persistence do ứng dụng sở hữu

Mô hình dữ liệu gồm `Meeting`, `TranscriptSegment`, `AnalysisRun`, `AnalysisUnit`, `SentimentPredictic` `MeetingSentimentWindow`, `IncomingWebhook` và `OutboxEvent`. Cách tách thực thể này cho phép một transcript gốc được giữ độc lập với các unit đã chuẩn hóa và prediction do nhiều model version tạo ra. Evidence không phải text sao chép vào dashboard mà là liên kết có thể truy ngược từ aggregate đến unit, segment và revision nguồn.

Segment được upsert theo `(meeting_id, vexa_segment_id)`. Khi text, speaker hoặc timestamp đổi, backend tăng revision của segment và `transcript_revision` của meeting. Analysis run được khóa duy nhất theo `(meeting_id, kind, source_revision, model_version)`. Các ràng buộc này giúp retry API/worker không tạo nhiều run tương đương và cho phép phân tích lại khi transcript hoặc model thay đổi. Kết quả cũ được supersede bằng `is_current=false`, thay vì bị xóa, để bảo toàn provenance [17].

5.3.2 Outbox, queue và khôi phục

Outbox event được ghi trong cùng domain dữ liệu PostgreSQL với segment hoặc kết quả phân tích. Event gồm loại event, meeting, payload và dedupe key duy nhất. Khi tạo final analysis run, backend commit run trước rồi enqueue định danh run vào Valkey Stream; nếu enqueue lỗi, run vẫn tồn tại ở trạng thái `queued`. Worker có cơ chế quét run queued để khôi phục khi Valkey tạm thời không sẵn sàng.

Worker xử lý message qua consumer group, chỉ acknowledge message sau khi xử lý/commit thành công. Khi lỗi, transaction được rollback, `analysis_run` được đánh dấu failed với error code và message không được acknowledge. Cơ chế này hỗ trợ retry có kiểm soát, nhưng không phải cam kết exactly-once end-to-end; retry vẫn đòi hỏi các unique constraint và idempotency key ở persistence.

5.4 Giao tiếp bất đồng bộ và delivery cho dashboard

Backend thực hiện hai kiểu giao tiếp. Kiểu request–response phục vụ các thao tác tạo, xem, đồng bộ, dừng meeting và đọc analytics. Kiểu event-driven phục vụ lifecycle transcript/analysis và delivery cập nhật. Worker tạo `analysis.sentiment.updated` sau khi aggregate được lưu; endpoint `/v1/meetings/{meeting_id}/events` đọc các outbox event theo meeting và phát định dạng `text/event-stream` kèm keepalive.

SSE là cơ chế server-to-client trên HTTP cho luồng event một chiều [15]. Trong MVP, SSE endpoint poll outbox mỗi giây và giữ tập event ID đã gửi trong vòng đời kết nối để tránh phát lặp trên cùng connection. Dashboard dùng event như tín hiệu refresh dữ liệu analytics; các giá trị KPI, timeline và evidence vẫn được đọc qua API. Điều này tránh để event payload trở thành nguồn dữ liệu độc lập nhưng đồng nghĩa với việc reconnect/replay, retention outbox và số lượng kết nối cần được thiết kế lại nếu triển khai quy mô lớn.

5.5 Bảo mật tại các boundary

5.5.1 Xác thực webhook và API key

Webhook từ Vexa được xác minh bằng HMAC-SHA256 trên chuỗi `timestamp.raw_body`; timestamp quá năm phút hoặc chữ ký không hợp lệ bị từ chối. HMAC là cơ chế xác thực thông điệp dựa trên hàm hash và secret dùng chung [23]. Event ID được lưu trong `incoming_webhooks`; nếu vi phạm khóa duy nhất, backend trả trạng thái duplicate thay vì chạy lại lifecycle meeting.

Vexa API key chỉ tồn tại trong cấu hình backend và được gửi server-to-server. Dashboard không nhận key, không gọi endpoint transcript của Vexa và không truy cập PostgreSQL/Valkey. Production cần dùng secret manager hoặc file quyền hạn chế, TLS reverse proxy, rotation secret và logging đã redaction; các cơ chế này là yêu cầu vận hành, chưa phải bằng chứng tuân thủ bảo mật của MVP.

5.5.2 Identity ứng dụng và kiểm soát truy cập

Trong development, header `X-App-User-Id` tạo identity tạm và chỉ khi `APP_ENV=development` mới có fallback `local-dev`. Đây không phải authentication production. Trước khi public service cần thay bằng session hoặc JWT/OAuth được xác minh, xác lập workspace/role, kiểm tra tenant isolation, rate limit và chính

sách audit. Việc tách owner check trong service layer giúp thay cơ chế identity mà không phải sửa mọi endpoint, nhưng không thay thế một thiết kế authorization đầy đủ.

5.6 Khả năng vận hành và hướng tích hợp mở rộng

Endpoint `/health` kiểm tra process sống, còn `/ready` chỉ trả ready khi PostgreSQL và Valkey đều phản hồi. Startup áp dụng compatibility upgrade và tạo schema; poller chạy nền reconcile các meeting đang active theo interval cấu hình. Với Vexa, webhook là đường notification, còn REST reconcile là fallback để giảm nguy cơ mất transcript khi event đến muộn hoặc delivery thất bại.

Kiến trúc hiện tại phù hợp cho MVP local, nơi FastAPI, worker, PostgreSQL, Valkey và dashboard chạy theo Docker Compose. Khi vận hành production, các thay đổi hợp lý gồm tách API/worker autoscaling, reverse proxy TLS, authentication thật, connection/pool tuning, queue monitoring, backup/restore, retention/delete workflow, alerting và load test. Các lựa chọn như nhiều microservice, direct Vexa WebSocket ingestion, distributed tracing, dynamic batching, ONNX, MLflow registry hoặc Kubernetes chỉ được triển khai sau khi có yêu cầu và benchmark tương ứng. Chúng không phải bằng chứng của MVP hiện tại.

Chương 6

Giao Diện Người Dùng và Dashboard

Frontend là lớp diễn giải kết quả pipeline cho người dùng, không phải một lớp suy luận độc lập. Thiết kế dashboard vì vậy phải đồng thời làm ba việc: trình bày trạng thái hệ thống, cho phép đối chiếu kết quả với evidence transcript, và truyền đạt giới hạn của output. Chương này mô tả single-page dashboard hiện có, cách nó tích hợp REST/SSE với backend và các yêu cầu thiết kế khi vận hành thực tế.

6.1 Mục tiêu thiết kế và ranh giới sử dụng

Dashboard biểu diễn output sentiment của transcript, không biểu diễn điểm rủi ro, năng lực, cảm xúc nội tại hay trạng thái sức khỏe tâm thần của participant. Một polarity aggregate chỉ có ý nghĩa trong phạm vi analysis unit, revision và model version đã tạo ra nó. Do đó, giao diện ưu tiên bốn nguyên tắc: *context* (có transcript/timestamp/speaker), *traceability* (có evidence và version), *uncertainty* (có coverage/confidence/unknown) và *control* (trạng thái tải/lỗi rõ ràng, không tự động tạo quyết định).

Các yêu cầu này biến dashboard thành một phần của cơ chế human oversight, thay vì một màn hình chỉ hiển thị metric. Confidence chỉ là điểm do adapter trả về và không được trình bày như xác suất về con người. Khi không có dữ liệu, coverage thấp hoặc label là *unknown*, giao diện cần bảo toàn trạng thái "không đủ dữ liệu" thay vì dùng giá trị mặc định trung tính.

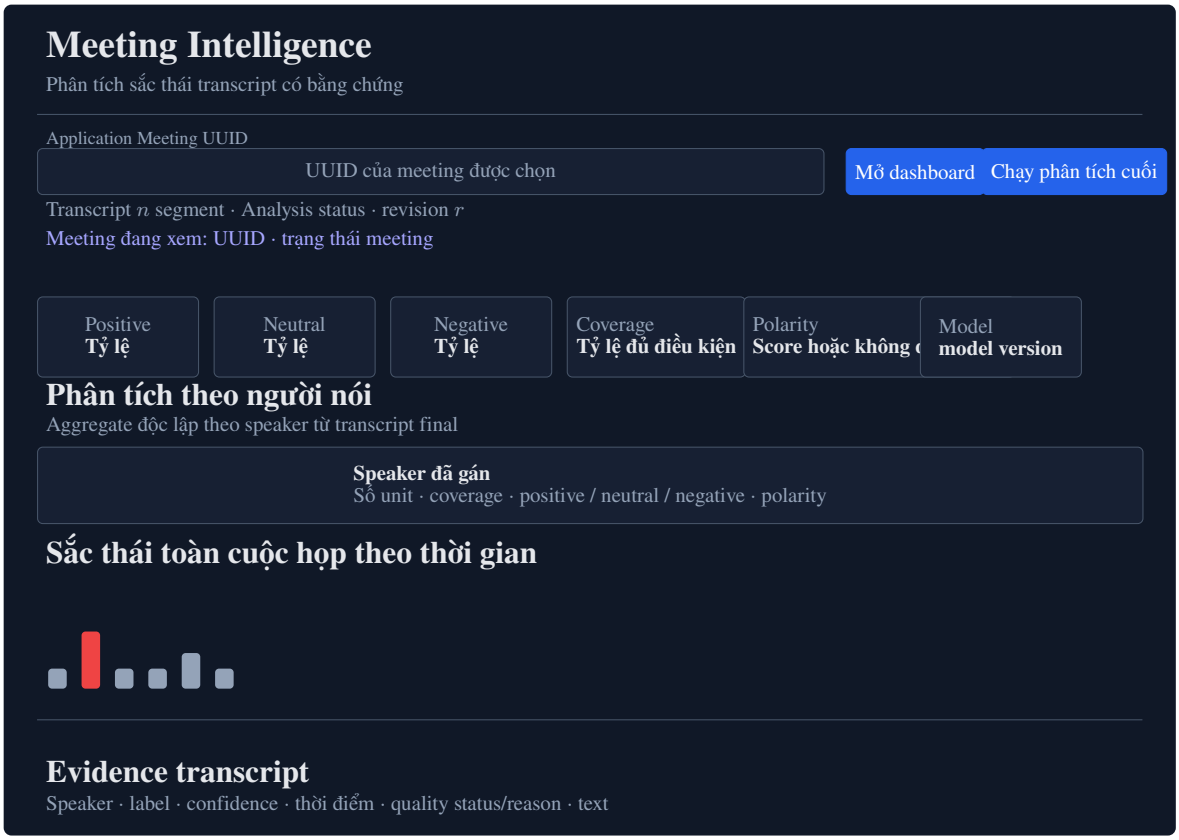
Bảng 6.1: Yêu cầu thiết kế dashboard và hiện thực trong MVP

Yêu cầu	Hiện thực hiện tại	Ràng buộc/việc cần làm
Khôi phục ngữ cảnh	Tự tải meeting mới nhất; cho phép nhập application meeting UUID hợp lệ	Chưa có danh sách/lọc meeting đầy đủ hay workspace selector
Quan sát tổng quan	KPI positive/neutral/negative, coverage, polarity và model version	Không diễn giải KPI thành đánh giá con người
Kiểm tra evidence	Hiển thị speaker, text, timestamp, quality status/reason, label và confidence	Chưa có lọc, phân trang hoặc export evidence
Cập nhật gần thời gian thực	`EventSource` nhận SSE; debounce refresh 300 ms	SSE hiện dựa trên polling outbox ở backend, chưa có replay cursor/notification UI
Phản hồi lỗi/trạng thái	Vùng `role=status` báo tải, meeting hiện hành và lỗi API	Chưa có error boundary, retry control hay offline state
Accessibility	Form, button, heading và status dùng phần tử semantic cơ bản	Chưa có audit WCAG, label/legend đầy đủ cho timeline hoặc kiểm thử assistive technology

6.2 Thiết kế không gian thông tin và mô hình giao diện

Giao diện được tổ chức theo thứ tự ra quyết định có thể kiểm tra: (i) chọn đúng application meeting; (ii) xác nhận trạng thái transcript, analysis run và revision; (iii) quan sát aggregate ở cấp cuộc họp; (iv) đặt aggregate vào lát cắt speaker hoặc thời gian; và cuối cùng (v) đối chiếu với evidence. Trình tự này ngăn một KPI được đọc tách khỏi nguồn dữ liệu của nó. Nó cũng tách rõ trạng thái điều khiển (UUID, nút mở dashboard, yêu cầu final analysis) khỏi trạng thái mô tả (coverage, model version, timeline và evidence), nhờ đó người dùng có thể nhận biết thao tác nào làm thay đổi hệ thống.

Hình 6.1 là mô hình giao diện được cập nhật theo dashboard trong mã nguồn và ảnh vận hành đại diện. Hình là *wireframe* phục vụ mô tả cấu trúc; UUID, tên speaker, tỷ lệ sentiment và transcript trong hình không phải dữ liệu cuộc họp thật. Tỷ lệ/nút chỉ trở nên có nghĩa khi gắn với `source_revision`, `model_version`, coverage và evidence mà backend trả về.



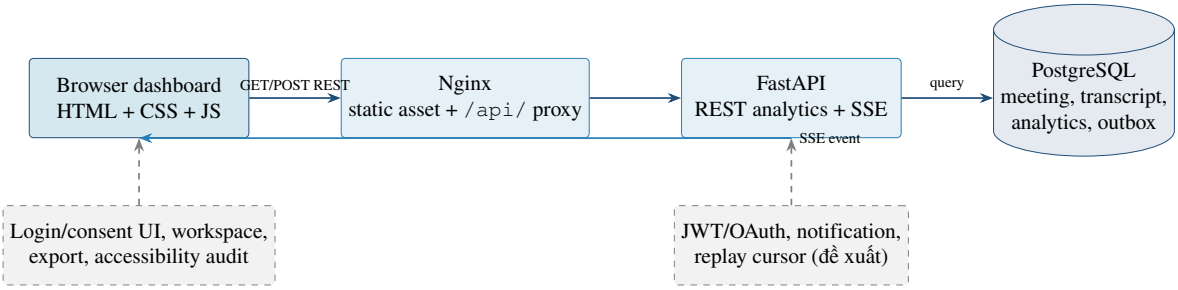
Hình 6.1: Mô hình giao diện dashboard của MVP. Bố cục được xây dựng từ cấu trúc HTML/CSS/JavaScript hiện có và một ảnh vận hành đại diện; các giá trị trong hình là placeholder để tránh công bố dữ liệu cuộc họp.

Sáu card overview dùng cùng một hàng để hỗ trợ so sánh nhanh, nhưng coverage và model version phải luôn đi cùng các tỷ lệ sentiment. Card theo speaker chỉ là lát cắt của transcript đã được gán speaker, không là thước đo đánh giá con người. Timeline dùng button để từng điểm có thể nhận focus/click; màu cần đi kèm title, legend và nhãn văn bản khi hoàn thiện accessibility, vì màu đơn lẻ không đủ để truyền đạt ý nghĩa [24].

6.3 Kiến trúc frontend và tích hợp backend

Dashboard được triển khai như static single-page application bằng HTML, CSS và JavaScript module. Nginx phục vụ asset, đặt header `X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy` và reverse proxy cùng origin từ đường dẫn `/api/` sang FastAPI. Browser không biết địa chỉ nội bộ backend, không chứa Vexa API key và không kết nối trực tiếp tới PostgreSQL, Valkey hay Vexa.

Hình 6.2 mô tả luồng tích hợp. REST là đường đọc dữ liệu có thẩm quyền; SSE chỉ là tín hiệu rằng dữ liệu có thể đã đổi. Sau khi nhận event, frontend đọc lại transcript/analytics từ API thay vì coi event payload là nguồn dữ liệu độc lập. SSE là cơ chế server-to-client một chiều trên HTTP [15], phù hợp với dashboard chủ yếu nhận thông báo cập nhật.



Hình 6.2: Kiến trúc frontend và các boundary tích hợp. Nét đứt: hạng mục chưa được triển khai/kiểm chứng.

6.4 Mô hình trạng thái và quá trình vận hành giao diện

Khi tải trang, frontend gọi `GET /v1/meetings?limit=1`. Nếu có meeting, ứng dụng kiểm tra UUID, lưu trạng thái cục bộ `meetingId/meeting`, rồi tải song song transcript, overview, speaker aggregates và timeline. Evidence được tải qua endpoint units sau khi các thành phần chính hoàn tất. Nếu người dùng nhập UUID, frontend chỉ gọi API khi chuỗi thỏa biểu thức UUID; điều này tránh request vô nghĩa đến backend do input lỗi.

Sau khi chọn meeting, frontend tạo `EventSource` đến endpoint event của meeting. Hai event `transcript.segment.` và `analysis.sentiment.updated` cùng gọi `scheduleRefresh`; timer debounce 300 ms gom các event liên tiếp trước khi reload dữ liệu. Người dùng có thể yêu cầu `POST /analysis?kind=final` bằng nút "Chạy phân tích cuối". Chuỗi trạng thái này được tóm tắt như sau:

`Chọn meeting → tải REST → render dashboard → kết nối SSE → event → refresh REST.`

Thiết kế refresh lại từ REST có ưu điểm là mọi card, timeline và evidence cùng phản ánh state hiện hành trong PostgreSQL, kể cả khi event bị gom hoặc đến sát nhau. Đổi lại, nó tạo thêm request sau event và chưa tối ưu cho số lượng dashboard lớn. Nếu mở rộng, frontend cần hỗ trợ reconnection status, `Last-Event-ID`/replay policy, exponential backoff, hủy request cũ và chỉ refresh vùng dữ liệu bị ảnh hưởng sau khi đã đo tải thực tế.

6.5 Các thành phần trình bày và diễn giải output

6.5.1 Tổng quan, timeline và speaker

Khu vực overview hiển thị sáu giá trị: tỷ lệ positive, neutral, negative; coverage; polarity; và model version. Polarity hiển thị "Không đủ dữ liệu" khi score là `null`. Khu vực speaker hiển thị unit count, coverage, tỷ lệ các lớp và polarity riêng theo speaker mà backend đã gán; khi không có final transcript gán speaker, giao diện hiển thị trạng thái trống thay vì suy đoán danh tính.

Timeline dùng các button theo cửa sổ thời gian. Màu xanh/lục, đỏ và xám hiện chỉ mã hóa polarity aggregate để hỗ trợ quét nhanh, không phải màu trạng thái của một người. Khi chọn một điểm timeline, dashboard tải evidence hiện hành của meeting. Ở phiên bản tiếp theo cần liên kết điểm đã chọn với chính cửa sổ thời gian tương ứng, thêm legend/nhãn văn bản và không chỉ dựa vào màu; hiện click timeline đang tải toàn bộ evidence thay vì lọc theo window.

6.5.2 Evidence và trạng thái không chắc chắn

Evidence hiển thị speaker, label, confidence, text, thời điểm, quality status và quality reasons. Việc dùng `textContent` và DOM API để render transcript/evidence giúp tránh chèn text nguồn như HTML. Nội dung transcript vẫn là dữ liệu nhạy cảm; giao diện chỉ nên cung cấp cho người có quyền và không được đưa vào URL, analytics bên thứ ba hoặc log browser không cần thiết.

Unknown, unit bị từ chối, coverage thấp và lỗi API là trạng thái hợp lệ của hệ thống. Giao diện hiện thể hiện một phần các trạng thái này qua text mô tả và `role=status`; khi vận hành thực tế cần bổ sung trạng thái loading/error riêng cho từng panel, retry có chủ đích, explanation ngắn cho coverage và cảnh báo về phạm vi diễn giải. Các cơ chế này quan trọng hơn việc che giấu dữ liệu thiếu bằng một chart "đẹp".

6.6 Accessibility, privacy và an toàn giao diện

HTML hiện dùng `form`, `label`, `button`, heading, section và vùng `role=status`; đây là nền tảng semantic cơ bản. WCAG 2.2 cung cấp các success criteria có thể kiểm tra, không phụ thuộc công nghệ, cho accessibility web [24]. Trong thiết kế tiếp theo, dashboard cần kiểm tra keyboard navigation/focus, tương phản, target size, thông báo thay đổi qua screen reader, label có thể đọc được của điểm timeline và cách biểu diễn không chỉ dựa vào màu. Các yêu cầu này là mục tiêu thiết kế, không phải tuyên bố dashboard đã tuân thủ WCAG.

Dashboard hiện bind cổng chỉ trên loopback qua Docker Compose và Nginx đặt một số security header. Tuy nhiên, nó chưa có login UI, consent UI, session timeout, content-security policy, CSRF strategy hoặc authorization production. Backend hiện dùng identity development; do đó giao diện không được public trước khi có authentication thật, TLS/reverse proxy, workspace authorization và kiểm thử tenant isolation. Privacy by design yêu cầu các kiểm soát này được xem xét xuyên suốt vòng đời xử lý dữ liệu [3].

6.7 Đánh giá UX và hướng tích hợp khi vận hành

Đánh giá giao diện cần trả lời liệu người dùng có hiểu đúng giới hạn output hay không, thay vì đo khả năng "phát hiện trầm cảm". Protocol đề xuất gồm các nhiệm vụ tìm evidence, nhận biết coverage thấp/unknown, so sánh revision/model version và diễn giải timeline có giới hạn; mỗi nghiên cứu cần consent, tiêu chí tuyển người tham gia, kịch bản dữ liệu an toàn và kế hoạch xử lý phản hồi. SUS có thể dùng như một thước đo usability ngắn [25]; NASA-TLX là một lựa chọn đo perceived workload [26]. Các score chỉ được báo cáo sau khi có nghiên cứu người dùng thực tế.

Các hướng tích hợp tiếp theo gồm: authentication/authorization thật và workspace selector; lifecycle consent trước khi hiển thị transcript; filter/pagination/export evidence có quyền hạn; notification/reconnect state; accessibility audit; i18n; và dashboard đa nền tảng sau khi backend có adapter tương ứng. Những hạng mục này không phải chức năng đã triển khai trong MVP và cần được kiểm thử bảo mật, UX và hiệu năng riêng.

Chương 7

Hạ Tầng và Vận Hành

Hạ tầng của hệ thống được thiết kế để tách ứng dụng Meeting Intelligence khỏi nền tảng Vexa cung cấp bot và transcript. Mục tiêu là làm cho dữ liệu, secret, lifecycle và khả năng khôi phục của từng miền được nhận diện rõ ràng. Chương này mô tả topology Docker Compose đã dùng cho MVP cục bộ, quy trình vận hành và các điều kiện cần có trước khi chuyển sang môi trường production.

7.1 Nguyên tắc thiết kế hạ tầng

Container cung cấp cách đóng gói và chạy ứng dụng có thể tái lập, nhưng đồng thời đưa vào các rủi ro về image, secret, network và isolation; các rủi ro này cần được quản trị trong toàn bộ vòng đời triển khai [27]. Trong dự án, các nguyên tắc hạ tầng được chọn như sau:

- **tách domain dữ liệu:** transcript/prediction của ứng dụng nằm trong PostgreSQL ứng dụng; database và object storage của Vexa không được xem là cùng một backup domain;
- **tối thiểu hóa bề mặt mạng:** chỉ dashboard và API debug bind loopback trong MVP; PostgreSQL, Valkey, Vexa admin API và Docker socket không public;
- **cấu hình ngoài source:** một file `settings.env` cục bộ cấp biến runtime cho hai Compose project, không được commit hoặc ghi log;
- **readiness trước routing:** backend chỉ ready khi cả PostgreSQL và Valkey phản hồi; dashboard phụ thuộc backend ready;
- **khôi phục dựa trên dữ liệu bền vững:** PostgreSQL là source of truth, còn Valkey chỉ hỗ trợ queue/fan-out và có cơ chế phục hồi run đang chờ.

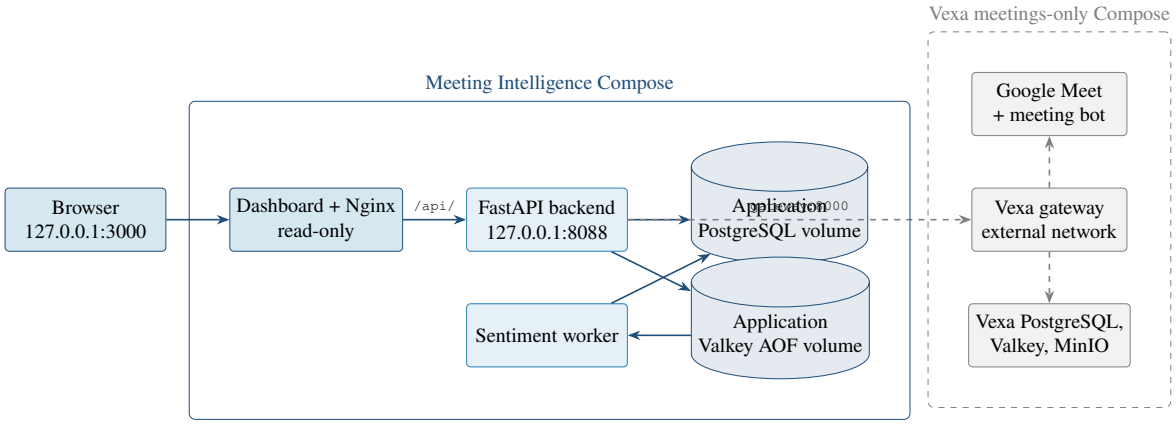
Các nguyên tắc này không tự động tạo ra high availability, disaster recovery hoặc compliance. Chúng là điểm xuất phát để xác định những gì cần kiểm chứng khi vận hành. Hướng dẫn contingency planning nhấn mạnh backup, recovery, kiểm thử và bảo trì là một phần của thiết kế hệ thống, không phải thao tác làm sau sự cố [28].

7.2 Topology Docker Compose đã kiểm chứng

7.2.1 Hai miền triển khai và ranh giới mạng

Hình 7.1 mô tả topology cục bộ. Compose của Meeting Intelligence gồm backend, worker, dashboard, PostgreSQL và Valkey. Compose Vexa meetings-only là một deployment riêng, gồm gateway, admin API, meeting API, runtime, PostgreSQL, Valkey và MinIO do Vexa sở hữu. Application Compose join external

network `vexa-meetings_vexa` để backend gọi Vexa gateway theo DNS Docker `gateway:8000`; điều này không có nghĩa các container của hai project chia sẻ volume, credential hoặc database.



Hình 7.1: Topology triển khai cục bộ. Nét đứt biểu thị kết nối giữa hai Compose project; các domain dữ liệu được backup riêng.

7.2.2 Dịch vụ, volume và health check

Bảng 7.1 tổng hợp thành phần của application Compose. Backend và worker dùng cùng image nhưng khác command, nhờ đó API không bị chặn bởi inference. Dashboard dùng Nginx phục vụ static asset, filesystem read-only và `tmpfs` cho thư mục runtime cần thiết. Valkey được bật append-only file (AOF), nhưng không thay thế PostgreSQL làm dữ liệu chuẩn.

Bảng 7.1: Các dịch vụ của Meeting Intelligence Compose

Dịch vụ	Vai trò	Persistence/điều kiện chạy
`backend`	FastAPI REST, SSE, webhook, reconcile	Chờ PostgreSQL và Valkey healthy; `/ready` kiểm cả hai store
`worker`	Đọc stream, quality gate, adapter và aggregate	Dùng cùng database/Valkey; tự phục hồi run `queued`
`dashboard`	Static frontend và Nginx reverse proxy `/api/`	Bind loopback; read-only filesystem, phụ thuộc backend healthy
`postgres`	Source of truth của application	Named volume `meeting-intelligence-postgres`
`valkey`	Stream/consumer group và fan-out hỗ trợ worker	Named volume `meeting-intelligence-valkey`, AOF bật

7.3 Cấu hình, secret và network

7.3.1 Một nguồn cấu hình runtime

`settings.env.example` là template được commit; `settings.env` là file cục bộ duy nhất cho cả application và Vexa self-host. Nhóm cấu hình ứng dụng gồm port, URL PostgreSQL, URL Valkey, model/preprocessing version, Vexa API base/key và webhook secret. Nhóm cấu hình Vexa gồm database, MinIO, control-plane secret, token transcription và network. Các credential của application PostgreSQL và Vexa PostgreSQL phải độc lập, không dùng cùng password hoặc volume.

Secret không được đưa vào image, dashboard, repository, log hay artifact của luận văn. Trong MVP local, file cấu hình cần quyền đọc hạn chế; khi production cần thay bằng secret manager, rotation, phân quyền runtime tối thiểu và quy trình thu hồi khi lộ lọt. Placeholder như `CHANGE_ME` không phải secret hợp lệ. API key Vexa chỉ cấp scope bot/transcript tối thiểu cần thiết và chỉ backend đọc.

7.3.2 Port exposure và giao tiếp giữa service

Dashboard và backend bind `127.0.0.1` trên host ở các port mặc định 3000 và 8088; các service data không publish port ra host trong Compose application. Nginx proxy `/api/` tới backend qua mạng Docker nội bộ. Vexa webhook cần một HTTPS URL mà Vexa có thể truy cập; `http://backend:8088` chỉ là DNS nội bộ của application và không mặc nhiên route được từ Compose Vexa. Production cần reverse proxy TLS phía trước, allow-list origin/rate limit phù hợp và không public database hoặc Docker socket.

7.4 Quy trình build, khởi động và kiểm chứng

Trình tự vận hành được tách thành Vexa và application. Trước hết, kiểm tra Docker Engine/Compose, tạo `settings.env`, cấu hình transcription và khởi động Vexa meetings-only. Sau khi Vexa gateway healthy, tạo API key đúng scope và đặt vào cấu hình application. Tiếp theo, `docker compose --env-file settings.env config` kiểm tra biến required/topology trước khi `up -d --build` tạo stack application.

Sau khởi động, script `verify.sh` kiểm tra `docker compose ps`, `/ready` và dashboard trên loopback. Script `runtime-status.sh` tạo biên bản read-only về container, port và readiness mà không in credential, transcript hoặc environment của container. Smoke pipeline dùng fixture transcript tổng hợp, chờ final analysis rồi xóa fixture; nó kiểm tra API–queue–worker nhưng không chứng minh chất lượng ASR hoặc sentiment. Các bước nghiệm thu đầy đủ cần bao gồm unit test, integration database/queue, dashboard SSE, Vexa contract, webhook valid/tampered/replay và E2E có consent.

Khi dừng stack thông thường, `docker compose down` giữ named volumes. Không dùng `down -v` cho application vì sẽ xóa PostgreSQL và Valkey volume. Việc xóa volume Vexa chỉ có thể được cân nhắc sau khi xác nhận target, backup và không còn cần database/MinIO/token của Vexa; nó là thao tác phá hủy, không phải bước khắc phục mặc định.

7.5 Quan sát, sao lưu và khả năng khôi phục

MVP có health check PostgreSQL/Valkey, `/health`, `/ready`, restart policy và cơ chế worker recovery. Các tín hiệu vận hành cần được theo dõi khi mở rộng gồm availability/readiness, API error rate, webhook verification failure, queue lag, run failed/retry, P50/P95 latency, database capacity, disk/volume usage và backup success/failure. Không có dashboard observability, alert rule hay SLO định lượng nào được tuyên bố đã triển khai.

Sao lưu phải tách tối thiểu thành application PostgreSQL, application Valkey (nếu policy yêu cầu), Vexa database và Vexa object storage. Backup chỉ có giá trị khi restore được thử trên môi trường tách biệt, có tiêu chí chấp nhận và không làm lộ transcript/secret. RTO/RPO là mục tiêu kinh doanh–kỹ thuật cần được xác định

từ BIA và phép thử khôi phục; chúng không thể được suy ra từ `restart: unless-stopped` hoặc việc có named volume [28].

7.6 Kiến trúc vận hành mở rộng và giới hạn

Đối với production, một kiến trúc mục tiêu có thể bổ sung TLS ingress, authentication/authorization, secret manager, image digest pinning, CI/CD có kiểm thử, backup/restore drill, structured log/metric/trace, policy retention/delete và monitoring. Worker có thể được scale độc lập sau khi đã đo throughput, queue lag, database pool và tài nguyên inference. Model artifact/GPU/MLflow chỉ được thêm khi có artifact, model registry và benchmark; không nên thêm GPU vào Compose chỉ để dự phòng.

Kubernetes, multi-region, active–active, HA database, autoscaling, cloud managed service, TimescaleDB, service mesh và SLO/RTO/RPO định lượng không thuộc implementation đã kiểm chứng. Tương tự, Zoom, Teams, Jitsi Meet, direct Vexa WebSocket và auto-alert cần contract, consent, capacity plan, security review và evaluation riêng. Sự phân biệt giữa topology local đã chạy và kiến trúc mục tiêu giúp tránh biến roadmap thành kết quả nghiên cứu.

Chương 8

Kiểm Chứng Ứng Dụng và Thiết Kế Thực Nghiệm

Chương này phân biệt hai loại bằng chứng có ý nghĩa khác nhau. *Kiểm chứng ứng dụng* trả lời liệu các thành phần hệ thống có hoạt động cùng nhau theo hợp đồng đã thiết kế hay không. *Đánh giá thực nghiệm* trả lời chất lượng của ASR, model sentiment, độ trễ, khả năng chịu tải hoặc trải nghiệm người dùng. MVP có test code và script phục vụ kiểm chứng integration; chưa có corpus transcript họp được gán nhãn, benchmark model, WER/CER, load test hoặc user study. Vì vậy, chương không công bố F1, AUC, accuracy, SLA hay kết luận về sức khỏe tâm thần.

8.1 Câu hỏi kiểm chứng và nguyên tắc báo cáo

Kiểm chứng hệ thống tập trung vào bốn câu hỏi: (i) transcript có được nhận, lưu và version đúng không; (ii) quality gate/unit builder/adapter có tạo output theo hợp đồng không; (iii) analysis run có đi qua queue–worker–persistence–aggregate không; và (iv) dashboard có nhận được dữ liệu/evidence cập nhật không. Các câu hỏi này phù hợp với đóng góp artefact của khóa luận nhưng không thay thế evaluation một bộ phân loại học máy.

Mỗi lần chạy cần lưu một evidence bundle tối thiểu: commit hoặc image digest; file cấu hình đã redaction; thời điểm chạy; phiên bản container; command/script; exit code; log không chứa transcript thật/secret; application meeting UUID (nếu có); và bản ghi analytics/evidence liên quan. Kết quả "pass" chỉ được ghi khi có artifact tương ứng. Một script tồn tại trong repository không được xem là test đã chạy.

Bảng 8.1: Ma trận kiểm chứng và trạng thái bằng chứng của MVP

Nhóm	Artefact/test hiện có	Tiêu chí pass	Phạm vi tuyên bố
Unit quality/unit builder	`test_quality.py`, `test_units.py`	Chuẩn hóa tiếng Việt; reject hallucination; ghép unit; bỏ duplicate gần nhau	Logic tiền xử lý, không đo chất lượng ASR/model
Unit service/Vexa parser	`test_service.py`, `test_vexa.py`	Fallback speaker có kiểm soát; URL Google Meet lỗi bị từ chối; upstream status được giữ	Hợp đồng helper, không chứng minh Vexa E2E
Integration pipeline	`smoke-analysis-pipeline.sh`	Fixture → API → queue → worker → analytics thành công	Luồng nội bộ với dữ liệu tổng hợp
Runtime reachability	`verify.sh`, `runtime-status.sh`	Compose hoạt động; `/ready` và dashboard phản hồi	Readiness/reachability, không phải load test
Vexa/webhook E2E	Kịch bản vận hành được mô tả	Bot, transcript, HMAC, replay và finalize được kiểm tra có log	Cần chạy thực tế với consent; chưa có số liệu báo cáo
Model/ASR/UX/performance	Chưa có dataset/protocol/run hoàn chỉnh	Xem các mục sau của chương	Chưa có kết quả thực nghiệm

8.2 Kiểm thử mức đơn vị và hợp đồng

Ở lần kiểm chứng cục bộ ngày 28/09/2026, lệnh `uv run pytest -q` hoàn tất 11/11 test case. Nhóm quality kiểm tra NFC/whitespace vẫn giữ dấu tiếng Việt, mẫu hallucination kiểu "subscribe" bị từ chối và dấu hiệu tiếng Việt có thể ưu tiên hơn language label upstream sai. Nhóm unit builder kiểm tra ghép ba final segment liền kề của cùng speaker, giữ segment bị từ chối như evidence không inferable và bỏ duplicate final segment phát gần nhau. Nhóm service kiểm tra chỉ dùng fallback speaker khi provider chưa gán speaker; nhóm Vexa parser kiểm tra URL Markdown không hợp lệ và bảo toàn status code lỗi upstream.

Các test này nên chạy trong môi trường backend khóa dependency, ví dụ `uv run pytest`, và log kết quả phải được lưu vào evidence bundle. Chúng kiểm tra invariant của code; không dùng transcript thật, không gọi Vexa và không cho biết một model nhận nhãn sentiment đúng bao nhiêu lần. Khi thêm model adapter, test contract cần bổ sung input tiếng Việt/ngoài ngôn ngữ, shape của probability, label mapping, failure mode khi artifact không tải được và tính nhất quán giữa `model_version` cấu hình với version thực tế.

8.3 Kiểm chứng integration và vận hành

8.3.1 Smoke pipeline tổng hợp

Script `smoke-analysis-pipeline.sh` tạo một meeting fixture với hai final segment tiếng Việt tổng hợp: một chứa marker tích cực và một chứa marker tiêu cực. Script tạo final analysis run qua API, chờ worker hoàn tất, đọc overview và units, rồi kiểm tra status `succeeded`, coverage bằng 1, có đúng hai unit và tập nhãn $\{positive, negative\}$. Tất cả row phụ thuộc được xóa qua khóa UUID fixture khi script kết thúc, kể cả khi lỗi.

Smoke test này là kiểm chứng tốt cho đường đi application-owned API–PostgreSQL–Valkey Stream–worker–analytics API và cho runtime `baseline-rules-v2`. Tuy nhiên, fixture được chọn để khớp luật baseline, nên kết quả không được dùng làm accuracy/F1, không đại diện cho ngôn ngữ họp và không chứng minh PhoBERT hoặc custom model. Script cũng không thay thế test SSE browser, benchmark đồng thời, resilience khi process chết hoặc Vexa transcript thực.

8.3.2 Readiness, dashboard và contract nguồn

`verify.sh` kiểm tra trạng thái Compose, endpoint `/ready` của backend và HTML dashboard trên loopback. `runtime-status.sh` tạo audit read-only về container/port/readiness nhưng không in `settings.env`, transcript hay biến môi trường container. Các script này nên chạy trước demo và sau thay đổi hạ tầng; output pass chỉ chứng minh service reachable tại thời điểm chạy.

Kịch bản contract Vexa cần kiểm tra độc lập: spawn bot đúng meeting/owner, đọc transcript, stop bot, webhook có chữ ký hợp lệ, webhook chữ ký sai, timestamp hết hạn, event ID trùng, retry reconcile và finalize. Mỗi trường hợp phải nêu input, response HTTP kỳ vọng, row database/event dự kiến và cách cleanup. Không dùng transcript cuộc họp thật trong Git, fixture hay CI log; E2E thật chỉ được thực hiện sau khi participant consent và endpoint HTTPS của webhook đã sẵn sàng.

8.4 Protocol đánh giá model sentiment

8.4.1 Dữ liệu, nhãn và split

Để đánh giá một model sentiment, cần corpus transcript họp đúng task với guideline ba lớp, trường hợp *unknown* và quy tắc cho phủ định, mỉa mai, code-switching, thiếu ngữ cảnh và trích dẫn lời người khác. Ít nhất hai annotator độc lập nên gán nhãn trước adjudication; agreement cần được báo cáo bằng Cohen's kappa hoặc Krippendorff's alpha cùng phân bố nhãn. Agreement không thay thế chất lượng model, nhưng là bằng chứng về độ tin cậy của nhãn tham chiếu [29].

Train/validation/test phải split theo **meeting**, không split ngẫu nhiên theo segment, để tránh speaker/topic/context leakage. Test set được khóa trước tuning. Dataset manifest ghi nguồn, consent/license, số meeting/speaker/unit, tiêu chí quality gate, date range, hash, preprocessing, label guideline và lý do loại dữ liệu. Không đưa dữ liệu ngoài domain hoặc notebook phân loại trầm cảm nhị phân vào cùng bảng kết quả với sentiment transcript.

8.4.2 Metric, uncertainty và so sánh model

Với lớp c, precision, recall và F1 được xác định:

$$P_c = \frac{TP_c}{TP_c + FP_c}, \qquad R_c = \frac{TP_c}{TP_c + FN_c}, \qquad F1_c = \frac{2P_cR_c}{P_c + R_c}. \tag{8.1}$$

Macro-F1 là trung bình không trọng số của $F1_c$ trên các lớp sentiment, vì vậy không để lớp neutral chiếm đa số che khuất lỗi ở positive/negative. Báo cáo đầy đủ gồm confusion matrix, precision/recall/F1 theo lớp, coverage, tỷ lệ *unknown*, calibration, latency P50/P95, phần cứng, batch size, seed và error analysis theo các lát dữ liệu đã

định trước.

Khi so sánh model, dùng cùng test set, quality gate, language routing, preprocessing policy và hardware. Confidence interval có thể được ước lượng bằng bootstrap với đơn vị resample là meeting, không phải segment, để tôn trọng cấu trúc phụ thuộc của dữ liệu [30]. Nếu tuyên bố model A tốt hơn B, cần nêu giả thuyết, phép kiểm định/interval, đơn vị phân tích và số lần thử; kiểm định ý nghĩa trong NLP phải phù hợp metric và thiết kế thực nghiệm [31].

Bảng 8.2: Mẫu báo cáo so sánh model sentiment (chỉ điền sau run có thể tái lập)

Model/artifact	Dataset/test version	Macro-F1 [CI]	Coverage	P95 inference
TF–IDF + Logistic Regression				
PhoBERT-base fine-tune				
PhoBERT-large fine-tune				
Custom model (nếu có)				

8.5 Đánh giá ASR, hệ thống và trải nghiệm người dùng

8.5.1 ASR và transcript

WER/CER cần audio gốc và transcript reference được chuẩn bị độc lập. Báo cáo phải cố định quy tắc tokenization tiếng Việt, điều kiện audio, loại meeting, noise/echo, speech overlap, code-switching và cách xử lý tên riêng. WER/CER là metric của ASR, không thể dùng thay F1 sentiment; ngược lại, F1 sentiment không cho biết transcript đã được nhận dạng chính xác bao nhiêu. MVP chưa có bộ dữ liệu này nên không báo cáo WER/CER.

8.5.2 Hiệu năng và resilience hệ thống

Khi đo hệ thống, cần ghi workload generator, số meeting/segment, phân bố kích thước text, concurrency, thời gian chạy, phần cứng/container image và cách warm-up. Độ trễ được tách:

$$T_{e2e} = T_{capture} + T_{ingest} + T_{queue} + T_{analysis} + T_{delivery}. \tag{8.2}$$

Metric tối thiểu gồm throughput, queue lag, success/failure/retry rate, P50/P95/P99 latency, resource profile và thời gian recovery sau khi restart backend/worker. Fault injection cần bao gồm Valkey unavailable, backend/worker restart, webhook duplicate/tampered/replay và transcript revision muộn. Không ghi SLA hay "real time" định lượng nếu chưa có workload và log đo tương ứng.

8.5.3 UX và cách diễn giải

UX study nên kiểm tra người dùng có tìm được evidence, nhận ra *unknown*/coverage thấp, phân biệt polarity transcript với đánh giá con người và hiểu model/revision hay không. Nhiệm vụ, participant criteria, consent, prototype/version, task completion, SUS/NASA-TLX và limitation phải được công bố. Không thiết kế UX study để suy luận ai có trầm cảm hoặc yêu cầu participant đưa ra quyết định nhân sự từ dashboard.

8.6 Biên bản bằng chứng cho một lần kiểm chứng integration

Mục này không phải là bảng kết quả sentiment. Nó là "biên bản truy xuất" cho một lần chạy integration, nhằm trả lời câu hỏi: kết luận "pipeline hoạt động" dựa trên lần chạy nào, ở phiên bản mã nào và có thể kiểm tra lại bằng artifact nào. Một run integration chỉ chứng minh luồng nội bộ API–PostgreSQL–Valkey–worker–analytics hoạt động với input đã nêu; nó không chứng minh Accuracy, Macro-F1, WER ASR hoặc SLA.

Mỗi run có một Evidence ID duy nhất, ví dụ `EVID-20260928-INT01`. Evidence bundle được lưu cục bộ trong `source/evidence/runs/<Evidence ID>/`; cấu trúc, cách redaction và script tạo bundle được mô tả trong `source/evidence/README.md`. Quy trình ghi nhận gồm bốn bước: (i) chạy script kiểm chứng; (ii) kiểm tra exit code và log; (iii) điền manifest bằng thông tin môi trường thực tế; và (iv) chỉ trích dẫn Evidence ID trong luận văn khi bundle có đầy đủ artifact. Nếu integration chưa chạy, phải ghi rõ "chưa đo" và không điền số liệu.

Bảng 8.3: Nội dung cần ghi trong evidence bundle của một run integration

Trường evidence	Nội dung cần ghi	Nguồn trong bundle
Định danh run	Evidence ID, ngày/giờ ICT, người chạy, kết quả PASS/FAIL	`manifest.md`
Phiên bản/môi trường	Git commit, OS, CPU/RAM, Docker/Compose, image digest nếu có	`environment.txt`, `manifest.md`
Lệnh và kết quả	Command đầy đủ, exit code, stdout/stderr	`commands.txt`, `exit-code.txt`, `stdout.log`
Dữ liệu và quyền riêng tư	Fixture ID đã redaction; synthetic hay real-consented; consent/retention áp dụng	`manifest.md`
Vòng đời pipeline	Transcript revision, analysis run status, accepted/rejected unit và lý do	API result đã redaction hoặc `stdout.log`
Coverage/version	Acceptance, prediction, effective coverage; `model_version`, `preprocessing_version`	Analytics API đã redaction, `manifest.md`
Kiểm tra UI/SSE	KPI, timeline, evidence, event SSE hoặc lý do chưa kiểm tra	Screenshot/API result đã redaction
Giới hạn/failure	Điều kiện bất thường, retry, điểm chưa đo và phạm vi claim được phép nêu	`manifest.md`

Ví dụ, Evidence ID của unit test chỉ được dùng để kết luận các invariant code đã pass; nó không thay thế Evidence ID integration. Sau khi có một bundle integration PASS, bảng kết quả ở phần sau hoặc phụ lục nên trích Evidence ID và tóm tắt số run/pass/fail, thay vì chép toàn bộ log vào luận văn.

8.7 Giới hạn thực nghiệm hiện tại

Rule baseline chỉ chứng minh integration khi smoke test pass; nó không thay thế model đã huấn luyện. Các notebook phân loại trầm cảm nhị phân ngoài domain hợp không được ngoại suy sang sentiment transcript do khác task, nhãn, dữ liệu và nguy cơ leakage. Chưa có số liệu coverage, accuracy, F1, WER, latency percentile, load, calibration hoặc UX được báo cáo như kết quả của khóa luận. Các transcript ngắn, multilingual, bị sửa

hoặc không đủ điều kiện tiếp tục được biểu diễn là *unknown*/rejected cùng lý do, thay vì bị diễn giải quá mức.

Chương 9

Quyền Riêng Tư, Đạo Đức và An Toàn

9.1 Mở đầu và phạm vi đánh giá

Hệ thống Meeting Intelligence xử lý transcript cuộc họp, siêu dữ liệu cuộc họp và các dự đoán cảm xúc ở mức *analysis unit*. Vì transcript có thể chứa tên riêng, phát biểu cá nhân, ý kiến nghề nghiệp hoặc thông tin nhạy cảm phát sinh trong ngữ cảnh họp, rủi ro không chỉ nằm ở rò rỉ dữ liệu mà còn ở việc diễn giải sai kết quả. Do đó, chương này xem đạo đức như một thuộc tính kiến trúc cần được thiết kế, kiểm thử và vận hành liên tục; không phải là một tuyên bố tuân thủ sau khi hệ thống đã hoàn thành. Cách tiếp cận này phù hợp với các nguyên tắc privacy-by-design [3] và trách nhiệm, minh bạch của hệ thống AI [32].

Phạm vi thực nghiệm hiện tại là luồng *Google Meet* → *Vexa* → *FastAPI/PostgreSQL/Valkey* → *worker* → *SSE Dashboard*. Vexa cung cấp sự kiện và bản sao transcript; dịch vụ của đề tài chỉ lưu dữ liệu cần thiết để phân tích, kiểm soát phiên bản transcript và trình bày kết quả. Bộ phân loại đang vận hành là luật xác định `baseline-rules-v2`; nó không phải công cụ chẩn đoán, không suy luận trạng thái tâm lý của cá nhân và không được dùng để xếp hạng nhân sự, đánh giá học tập, kỷ luật hoặc ra quyết định tự động. Các bộ tích hợp Teams, Zoom, Jitsi và mô hình học máy tùy biến là hướng mở rộng, không phải năng lực đã triển khai.

9.2 Mô hình rủi ro và nguyên tắc thiết kế

Ba lớp đối tượng cần được phân biệt rõ: (i) dữ liệu gốc, gồm transcript, người nói, thời điểm và trạng thái hiệu đính; (ii) dữ liệu dẫn xuất, gồm analysis unit, nhãn cảm xúc, confidence, evidence và chỉ số tổng hợp; (iii) quyết định của con người dựa trên dashboard. Nhãn ở lớp (ii) có thể sai do ASR, ngôn ngữ pha trộn, ngữ cảnh thiếu hoặc quy tắc baseline có độ bao phủ thấp. Vì vậy, một nhãn cảm xúc không được nâng thành kết luận về ý định, năng lực hay sức khỏe của người nói.

Các yêu cầu thiết kế được chuẩn hóa như sau.

- **Giới hạn mục đích:** chỉ phân tích nội dung transcript để hỗ trợ người được ủy quyền xem lại cuộc họp. Mục đích mới, chẳng hạn chấm điểm nhân viên hay theo dõi hành vi, phải được đánh giá lại về pháp lý, đạo đức và kỹ thuật trước khi sử dụng.
- **Tối thiểu hóa dữ liệu:** không chủ đích lưu audio/video thô trong dịch vụ phân tích; chỉ giữ transcript, siêu dữ liệu và evidence tối thiểu cần cho truy vết. Mọi trường dữ liệu phải có lý do xử lý và thời hạn lưu tương ứng.
- **Minh bạch và khả năng phản biện:** dashboard cần hiển thị coverage, trạng thái `unknown`, evidence, revision transcript, preprocessing và `model_version`. Người dùng không nên chỉ nhận một điểm số

tổng hợp không có ngữ cảnh.

- **Giám sát bởi con người:** prediction là thông tin hỗ trợ, không kích hoạt hành động bất lợi hay cảnh báo cá nhân tự động. Người đọc phải kiểm tra transcript và ngữ cảnh trước khi đưa ra nhận định.
- **Trách nhiệm giải trình:** phiên bản mô hình, cấu hình tiền xử lý, nguồn sự kiện và kết quả worker được lưu để tái lập một lần chạy. Đây là nền tảng kỹ thuật cho audit, sửa dữ liệu và điều tra sự cố, nhưng chưa tự động tạo ra sự tuân thủ pháp lý.

Các nguyên tắc này tương thích với yêu cầu về mục đích cụ thể, mức dữ liệu phù hợp và trách nhiệm của bên kiểm soát dữ liệu trong Nghị định 13/2023/NĐ-CP [33]; đồng thời có tính tham chiếu với GDPR [4]. Việc áp dụng cho một tổ chức cụ thể vẫn cần tư vấn pháp lý và đánh giá tác động bảo vệ dữ liệu (DPIA), đặc biệt khi phạm vi xử lý, quốc gia triển khai hoặc đối tượng sử dụng thay đổi.

9.3 Consent, quản trị dữ liệu và quyền của chủ thể dữ liệu

Việc tham dự một cuộc họp không mặc nhiên đồng nghĩa với đồng ý cho việc sao chép transcript, phân tích cảm xúc và lưu kết quả dẫn xuất. Trước khi kết nối nguồn dữ liệu, bên triển khai cần thông báo bằng ngôn ngữ dễ hiểu về nguồn thu thập, mục đích phân tích, loại dữ liệu, nơi lưu trữ, đối tượng có quyền truy cập, thời hạn lưu, đầu mối liên hệ và cách rút hoặc thay đổi sự đồng ý. Consent cần được ghi nhận tách biệt với thông báo lịch họp; khi cơ sở xử lý không phải consent, cơ sở đó cũng cần được xác định và rà soát theo thẩm quyền pháp lý phù hợp [33].

Trong nghiên cứu, dữ liệu thật không nên được đưa vào Git, tài liệu công khai, log CI/CD hoặc bộ kiểm thử. Các smoke test của đề tài sử dụng fixture tổng hợp và xóa dữ liệu kiểm thử sau khi xác nhận kết quả. Nếu cần dùng transcript thật để đánh giá, cần có quy trình tuyển chọn, khử định danh, phân quyền và phê duyệt đạo đức riêng. Người vận hành cũng cần xác định rõ vai trò giữa tổ chức sở hữu cuộc họp, nhà cung cấp nền tảng họp, Vexa và dịch vụ phân tích; không giả định rằng điều khoản của một nền tảng tự động bao phủ toàn bộ chuỗi xử lý tiếp theo.

9.4 Kiểm soát dữ liệu theo vòng đời hệ thống

Thiết kế hiện tại tạo một số điểm kiểm soát hữu ích, nhưng các điểm đó không thay thế một chương trình bảo vệ dữ liệu hoàn chỉnh.

Thu thập và nhập liệu. FastAPI nhận webhook Vexa có kiểm tra HMAC SHA-256, timestamp và cơ chế chống gửi lặp; dashboard không nắm API key của Vexa. Chỉ adapter Google Meet đang có trong mã nguồn. Khi bổ sung Teams, Zoom hoặc Jitsi, mỗi adapter phải được đánh giá lại về scope OAuth, webhook, dữ liệu nhận được và điều khoản nền tảng.

Lưu trữ và xử lý. PostgreSQL là nguồn dữ liệu chuẩn cho meeting, revision transcript, analysis run và evidence; Valkey chỉ dùng cho hàng đợi/phát tán công việc. Worker chỉ phân tích segment cuối cùng, không bị thu hồi và đã qua quality gate. Việc bỏ các đoạn không đủ điều kiện hoặc gán `unknown` làm giảm nguy cơ tạo nhãn giả, nhưng không xóa bỏ thiên lệch từ ASR hay dữ liệu đầu vào.

Truy cập và trình bày. Dashboard nhận cập nhật qua SSE; trong cấu hình phát triển, backend và dashboard được bind vào loopback. Cơ chế định danh hiện có `X-App-User-Id` chỉ phục vụ phát triển, chưa phải authentication/authorization đủ cho môi trường thật. Triển khai thực tế cần TLS đầu cuối, SSO/OIDC hoặc cơ chế tương đương, RBAC theo workspace/owner, quản lý secret, audit log và kiểm soát xuất dữ liệu.

Lưu giữ và xóa. Dự án đã lưu revision và evidence để truy vết, nhưng chưa triển khai retention schedule, workflow export/correction/deletion, xóa bản sao lưu hay chứng minh xóa dữ liệu. Đây là các hạng mục bắt buộc phải được thiết kế và kiểm thử trước khi xử lý dữ liệu thật lâu dài; không được suy diễn rằng việc không lưu audio/video đồng nghĩa không còn rủi ro từ transcript.

9.5 Công bằng, độ tin cậy và diễn giải kết quả

Độ chính xác của suy luận cảm xúc phụ thuộc vào chất lượng nhận dạng tiếng nói, phương ngữ, chuyển mã ngôn ngữ, thuật ngữ chuyên môn, số người nói và phần ngữ cảnh còn thiếu. Một mô hình hoặc rule được đánh giá tốt trên một tập dữ liệu không tự động công bằng với các nhóm người dùng khác; việc báo cáo chỉ một accuracy chung có thể che khuất sai số theo ngôn ngữ hoặc điều kiện âm thanh. Do đó, khi triển khai mô hình học máy tùy biến trong tương lai, tập đánh giá cần được mô tả bằng *datasheet* [13], báo cáo theo lát cắt phù hợp và xem xét calibration, coverage, false positive/false negative thay vì chỉ tối ưu một chỉ số tổng hợp.

Ở MVP, `baseline-rules-v2` được lựa chọn vì tính xác định và dễ audit, không phải vì đã chứng minh vượt trội về chất lượng. `Confidence` của output không được diễn giải như xác suất một người "có cảm xúc tiêu cực"; `unknown` và low coverage là kết quả hợp lệ cần hiển thị rõ. Dashboard không được biến tổng hợp theo speaker thành bảng xếp hạng, tín hiệu giám sát hoặc cơ sở cho quyết định có hậu quả. Các giới hạn này giúp tránh vòng phản hồi, hiệu ứng bị giám sát và việc dùng hệ thống vượt quá mục tiêu nghiên cứu ban đầu.

9.6 Ranh giới với an toàn và sức khỏe tâm thần

Sentiment trong transcript khác với sàng lọc hoặc chẩn đoán lâm sàng. Những công cụ như PHQ-9 có quy trình thẩm định riêng [5], còn tiêu chuẩn chẩn đoán yêu cầu đánh giá chuyên môn và ngữ cảnh lâm sàng [6]. Vì vậy, hệ thống không được suy luận trầm cảm, nguy cơ tự hại hay tình trạng sức khỏe tâm thần từ nhãn cảm xúc, keyword hoặc confidence. Không có tính năng cảnh báo khủng hoảng tự động trong phạm vi đề tài.

Nếu nghiên cứu trong tương lai mở rộng sang tự hại hoặc suicidality, công việc phải dừng trước khi thu thập/triển khai cho đến khi có phê duyệt ethics, người phụ trách chuyên môn lâm sàng, quy trình phản ứng khẩn cấp, cơ chế chuyển tuyến, đánh giá false alarm/missed detection và bảo vệ người tham gia. Keyword alert đơn thuần không phải là can thiệp an toàn.

9.7 Trạng thái hiện tại và điều kiện trước triển khai thực tế

MVP đã có provenance cho prediction (segment ID, revision, timestamp, preprocessing version và model version), quality gate, webhook signing, tách dashboard khỏi Vexa key và nguyên tắc không tự động ra quyết định. Tuy nhiên, hệ thống **chưa** tuyên bố tuân thủ GDPR, EU AI Act, Nghị định 13/2023/NĐ-CP hay bất kỳ khuôn

khổ nào khác; chưa có DPIA, review pháp lý, consent-management, RBAC/SSO sản xuất, retention/deletion workflow, audit trail hoàn chỉnh hoặc đánh giá fairness trên dữ liệu thật.

Trước khi vận hành ngoài phạm vi MVP, nhóm triển khai cần tối thiểu: xác lập data controller/processor và mục đích xử lý; hoàn tất DPIA và chính sách consent/retention; kiểm thử phân quyền, TLS, backup/restore và quy trình xóa; công bố hướng dẫn sử dụng cùng ranh giới diễn giải; lập quy trình báo cáo sự cố, phản hồi khiếu nại và đánh giá định kỳ model/data drift. Các điều kiện này biến nguyên tắc đạo đức thành tiêu chí nghiệm thu hệ thống có thể kiểm chứng, thay vì chỉ là cam kết trong tài liệu.

Các ràng buộc trên cũng định nghĩa phạm vi của kết luận nghiên cứu ở Chương 10: MVP được kết luận về thiết kế và kiểm chứng luồng xử lý có evidence, không được kết luận về hiệu quả lâm sàng, chất lượng sentiment trên dữ liệu thật hay mức độ tuân thủ pháp lý. Việc liên kết yêu cầu đạo đức với tiêu chí nghiệm thu kỹ thuật giúp bảo đảm mỗi năng lực mở rộng đều có thể được đánh giá về lợi ích, rủi ro và bằng chứng vận hành trước khi được đưa vào sử dụng.

Chương 10

Kết Luận và Hướng Phát Triển

10.1 Tổng kết nghiên cứu

Khóa luận tiếp cận bài toán phân tích sentiment trên transcript họp như một bài toán *thiết kế, hiện thực và kiểm chứng artefact hệ thống*, thay vì chỉ huấn luyện một bộ phân loại. Cách tiếp cận này phù hợp với nghiên cứu khoa học thiết kế trong hệ thống thông tin: artefact phải giải quyết một lớp vấn đề xác định và được đánh giá bằng bằng chứng phù hợp với mục tiêu của nó [7]. Trong phạm vi đó, đề tài đã xây dựng MVP cho luồng *Google Meet → Vexa → FastAPI/PostgreSQL/Valkey → worker → SSE Dashboard*.

Đóng góp chính của artefact là xác lập một đường đi dữ liệu có thể truy vết: transcript được tiếp nhận và version hóa; quality gate và language routing quyết định dữ liệu có đủ điều kiện suy luận; các segment được ghép thành analysis unit có ranh giới speaker/thời gian; worker tách khỏi API thực hiện phân tích bất đồng bộ; PostgreSQL lưu nguồn dữ liệu chuẩn, run, prediction và evidence; còn dashboard nhận aggregate/evidence qua SSE. Việc gắn prediction với segment ID, transcript revision, preprocessing version và model version làm cho một kết quả có thể được kiểm tra lại khi input hoặc cấu hình thay đổi. Đây là đóng góp về khả năng audit và tích hợp, không phải tuyên bố rằng mọi dự đoán đều đúng.

Runtime đã hiện thực là `baseline-rules-v2`, đặt sau classifier adapter để kiểm tra hợp đồng xử lý đầu cuối một cách xác định và dễ truy vết. Thiết kế adapter giữ chỗ cho PhoBERT hoặc custom model trong tương lai, nhưng PhoBERT không phải kết quả đã đạt của MVP. Tương tự, chỉ Google Meet thông qua Vexa là adapter có trong mã nguồn; Teams, Zoom và Jitsi mới là phương án kiến trúc mở rộng. Sự phân biệt giữa năng lực đã triển khai và phương án đề xuất là cần thiết để tránh nợ kỹ thuật và các tuyên bố vượt quá bằng chứng thực nghiệm [18].

10.2 Bằng chứng và phạm vi kết luận

Chương 8 đã tách kiểm chứng phần mềm khỏi đánh giá khoa học của ASR, mô hình và trải nghiệm người dùng. Test đơn vị kiểm tra các invariant của quality gate, unit builder, service và Vexa parser; smoke pipeline với fixture tổng hợp kiểm tra đường đi API–PostgreSQL–Valkey–worker–analytics cho hai nhãn luật baseline; các script readiness kiểm tra khả năng tiếp cận service. Những bằng chứng này hỗ trợ kết luận rằng kiến trúc MVP có cơ chế kiểm chứng integration và lưu vết phù hợp với mục tiêu artefact.

Ngược lại, khóa luận không báo cáo và không suy diễn F1/accuracy sentiment, WER/CER ASR, calibration, latency percentile, throughput, SLA, khả năng chịu tải, hiệu quả PhoBERT hay custom model. Không có corpus transcript họp được gán nhãn, benchmark đa mô hình, load test hoặc user study hoàn chỉnh. Vì vậy, các bảng kết quả định lượng được giữ là biểu mẫu; chỉ được điền sau một run có commit/image, manifest dữ liệu, cấu hình

redaction, log và evidence bundle tương ứng. Đây là điều kiện để kết quả sau này có thể tái lập và phản biện.

10.3 Ý nghĩa đạo đức và vận hành

Giá trị của hệ thống không nằm ở một điểm sentiment tổng hợp mà ở khả năng đặt điểm đó trong ngữ cảnh và thể hiện sự không chắc chắn. `unknown`, rejected unit, coverage thấp và evidence không phải lỗi giao diện cần che giấu, mà là các tín hiệu giúp người dùng tránh diễn giải quá mức. Theo Chương 9, sentiment của transcript không phải suy luận về năng lực, hành vi hay sức khỏe tâm thần của người nói. Dashboard không được dùng cho xếp hạng nhân sự, đánh giá học tập, kỷ luật hay quyết định tự động.

MVP đã có một số kiểm soát kỹ thuật ban đầu như webhook HMAC, deduplication, provenance và tách dashboard khỏi Vexa key. Tuy nhiên, các kiểm soát đó không đồng nghĩa hệ thống đã tuân thủ quy định. Trước khi xử lý dữ liệu thật trong môi trường vận hành, cần hoàn tất DPIA/review pháp lý, consent management, authentication và authorization sản xuất, TLS, retention/deletion, audit log, backup/restore và quy trình ứng phó sự cố. Privacy-by-design cần được duy trì suốt vòng đời hệ thống [3]; model card và tài liệu vận hành cũng cần ghi rõ mục đích, giới hạn, dữ liệu đánh giá và trường hợp không nên sử dụng [12].

10.4 Hạn chế

Các giới hạn quan trọng của nghiên cứu gồm:

- **Giới hạn dữ liệu và mô hình:** chưa có dataset sentiment đúng miền họp, protocol gán nhãn, split theo meeting, benchmark hoặc artifact model được promote vào serving. Các notebook phân loại trầm cảm nhị phân ngoài miền họp không phải bằng chứng cho sentiment transcript và không được ngoại suy sang sàng lọc lâm sàng.
- **Giới hạn nguồn và tích hợp:** chưa có adapter vận hành cho Teams, Zoom, Jitsi; chưa có E2E production với transcript thật có consent, endpoint HTTPS công khai và bằng chứng vận hành dài hạn.
- **Giới hạn hạ tầng:** Docker Compose hiện phục vụ MVP cục bộ; chưa chứng minh high availability, disaster recovery, autoscaling, quan sát tập trung hoặc resilience dưới tải và fault injection.
- **Giới hạn bảo vệ dữ liệu:** định danh bằng `X-App-User-Id` là cơ chế phát triển; hệ thống chưa có SSO/RBAC production, consent/retention/deletion workflow và đánh giá fairness trên dữ liệu thực.

10.5 Hướng phát triển có thể kiểm chứng

Hướng mở rộng nên được triển khai theo các mốc có điều kiện nghiệm thu rõ ràng, thay vì thay thế baseline bằng một model phức tạp mà không có bằng chứng.

1. **Chuẩn hóa dữ liệu và đánh giá:** xây dựng corpus transcript họp có consent, guideline nhãn, annotator độc lập, split theo meeting và test set khóa. Báo cáo datasheet, phân bố lớp, agreement, Macro-F1 theo lớp, coverage, calibration và error analysis; không dùng test set để tuning [13, 29].

2. **Vòng đời model:** đóng gói artifact cùng tokenizer, preprocessing, label mapping, threshold, dependency lockfile, evaluation report và model card. MLflow tracking/registry, shadow run và promotion policy là kiến trúc đề xuất để so sánh ứng viên với `baseline-rules-v2`, không phải service đang vận hành trong MVP.

3. **Đo lường hệ thống:** thiết kế workload đại diện, đo latency P50/P95/P99, throughput, queue lag, retry/failure rate, tài nguyên và recovery sau sự cố. Kết quả phải ghi rõ hardware, image, cấu hình và thời điểm chạy trước khi công bố SLA hoặc đặc tính "gần thời gian thực" định lượng.

4. **Mở rộng tích hợp an toàn:** thêm Teams, Zoom hoặc Jitsi qua adapter contract thống nhất; với mỗi nguồn, kiểm tra OAuth/webhook scope, signature, retry, revision, quyền dữ liệu và consent trước khi nối vào pipeline.

5. **Vận hành có trách nhiệm:** hoàn thiện quyền truy cập, retention/deletion, audit, backup/restore, quy trình khiếu nại/sự cố và đánh giá drift/fairness định kỳ. Mọi mở rộng sang safety hoặc sức khỏe tâm thần phải là nghiên cứu riêng có ethics approval, giám sát lâm sàng và protocol phản ứng phù hợp.

10.6 Lời kết

MVP cho thấy một hệ thống phân tích transcript có trách nhiệm phải quản lý đồng thời dữ liệu, ngữ cảnh, bất định và khả năng truy vết; việc thay bộ phân loại chỉ là một phần của bài toán. Nền tảng đã xây dựng tạo điều kiện cho các thực nghiệm NLP tiếng Việt trong họp trực tuyến ở bước tiếp theo, với nguyên tắc không biến tín hiệu ngôn ngữ thành kết luận về con người và không biến thiết kế đề xuất thành kết quả đã được chứng minh.

Tài liệu tham khảo

[1] D. Q. Nguyen and A. T. Nguyen, "PhoBERT: Pre-trained language models for vietnamese," in *Findings of EMNLP 2020*, 2020, pp. 1037–1042. [Online]. Available: https://aclanthology.org/2020.findings-emnlp.92/

[2] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," *arXiv preprint arXiv:2212.04356*, 2022. [Online]. Available: https://arxiv.org/abs/2212.04356

[3] A. Cavoukian, "Privacy by design: The 7 foundational principles," Information & Privacy Commissioner of Ontario, Tech. Rep., 2011. [Online]. Available: https://www.ipc.on.ca/wp-content/uploads/Resources/7foundationalprinciples.pdf

[4] European Parliament and Council, "Regulation (EU) 2016/679 (general data protection regulation)," Official Journal of the European Union, Tech. Rep. L119, 2016. [Online]. Available: https://eur-lex.europa.eu/eli/reg/2016/679/oj

[5] K. Kroenke, R. L. Spitzer, and J. B. W. Williams, "The PHQ-9: Validity of a brief depression severity measure," *Journal of General Internal Medicine*, vol. 16, no. 9, pp. 606–613, 2001. [Online]. Available: https://doi.org/10.1046/j.1525-1497.2001.016009606.x

[6] American Psychiatric Association, *Diagnostic and Statistical Manual of Mental Disorders*, 5th ed. Arlington, VA: American Psychiatric Publishing, 2013. [Online]. Available: https://www.psychiatry.org/psychiatrists/practice/dsm

[7] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004. [Online]. Available: https://doi.org/10.2307/25148625

[8] B. Pang and L. Lee, "Opinion mining and sentiment analysis," *Foundations and Trends in Information Retrieval*, vol. 2, no. 1–2, pp. 1–135, 2008. [Online]. Available: https://doi.org/10.1561/1500000011

[9] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html

[10] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of NAACL-HLT 2019*, 2019, pp. 4171–4186. [Online]. Available: https://aclanthology.org/N19-1423/

[11] A. Conneau, K. Khandelwal, N. Goyal, V. Chaudhary, G. Wenzek, F. Guzmán, É. Grave, M. Ott, L. Zettlemoyer, and V. Stoyanov, "Unsupervised cross-lingual representation learning at scale," in *Proceedings of ACL 2020*, 2020. [Online]. Available: https://aclanthology.org/2020.acl-main.747/

[12] M. Mitchell, S. Wu, A. Zaldivar, P. Barnes, L. Vasserman, B. Hutchinson, E. Spitzer, I. D. Raji, and T. Gebru, "Model cards for model reporting," in *Proceedings of the Conference on Fairness, Accountability, and Transparency*, 2019, pp. 220–229. [Online]. Available: https://doi.org/10.1145/3287560.3287596

[13] T. Gebru, J. Morgenstern, B. Vecchione, J. W. Vaughan, H. Wallach, H. Daumé, and K. Crawford, "Datasheets for datasets," *Communications of the ACM*, vol. 64, no. 12, pp. 86–92, 2021. [Online]. Available: https://doi.org/10.1145/3458723

[14] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proceedings of the 34th International Conference on Machine Learning*, vol. 70, 2017, pp. 1321–1330. [Online]. Available: https://proceedings.mlr.press/v70/guo17a.html

[15] World Wide Web Consortium, "Server-sent events," W3C Recommendation, Tech. Rep., 2015. [Online]. Available: https://www.w3.org/TR/2015/REC-eventsource-20150203/

[16] European Commission, "Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (artificial intelligence act)," Official Journal of the European Union, Tech. Rep., 2024. [Online]. Available: https://eur-lex.europa.eu/eli/reg/2024/1689/oj

[17] P. Buneman, S. Khanna, and W.-C. Tan, "Why and where: A characterization of data provenance," in *Database Theory — ICDT 2001*, 2001, pp. 316–330. [Online]. Available: https://doi.org/10.1007/3-540-44503-X_1

[18] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems*, vol. 28, 2015. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2015/file/86df7dcfd896fcaf2674f757a2463eba-Paper.pdf

[19] H. T. L. Song *et al.*, "ViHealthBERT: Pre-trained language models for Vietnamese in health text mining," in *Proceedings of LREC 2022*, 2022. [Online]. Available: https://aclanthology.org/2022.lrec-1.35

[20] M. Zaharia, A. Chen, A. Davidson, A. Ghodsi, S. A. Hong, A. Konwinski, S. Murching, T. Nykodym, P. Ogilvie, M. Parkhe, F. Xie, and C. Zumar, "Accelerating the machine learning lifecycle with MLflow," *IEEE Data Engineering Bulletin*, vol. 41, no. 4, pp. 39–45, 2018. [Online]. Available: http://sites.computer.org/debull/A18dec/p39.pdf

[21] MLflow Project, "Model registry workflows," MLflow Documentation, 2026, truy cập ngày 28/09/2026. [Online]. Available: https://mlflow.org/docs/latest/ml/model-registry/workflow

[22] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, University of California, Irvine, 2000. [Online]. Available: https://www.ics.uci.edu/~fielding/pubs/dissertation/fielding_dissertation.pdf

[23] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-hashing for message authentication," Internet Engineering Task Force, RFC 2104, 1997. [Online]. Available: https://www.rfc-editor.org/rfc/rfc2104

[24] World Wide Web Consortium, "Web content accessibility guidelines (WCAG) 2.2," W3C Recommendation, Tech. Rep., 2024. [Online]. Available: https://www.w3.org/TR/2024/REC-WCAG22-20241212/

[25] J. Brooke, "SUS: A "quick and dirty" usability scale," in *Usability Evaluation in Industry*. Taylor & Francis, 1996, pp. 189–194. [Online]. Available: https://www.taylorfrancis.com/chapters/edit/10.1201/9781498710411-35/sus-quick-dirty-usability-scale-john-brooke

[26] S. G. Hart and L. E. Staveland, "Development of NASA-TLX (task load index): Results of empirical and theoretical research," in *Advances in Psychology*. North-Holland, 1988, vol. 52, pp. 139–183. [Online]. Available: https://doi.org/10.1016/S0166-4115(08)62386-9

[27] M. Souppaya, J. Morello, and K. Scarfone, "Application container security guide," National Institute of Standards and Technology, NIST Special Publication 800-190, 2017. [Online]. Available: https://csrc.nist.gov/pubs/sp/800/190/final

[28] M. Swanson, P. Bowen, A. Phillips, D. Gallup, and D. Lynes, "Contingency planning guide for federal information systems," National Institute of Standards and Technology, NIST Special Publication 800-34 Revision 1, 2010. [Online]. Available: https://csrc.nist.gov/pubs/sp/800/34/r1/upd1/final

[29] R. Artstein and M. Poesio, "Survey article: Inter-coder agreement for computational linguistics," *Computational Linguistics*, vol. 34, no. 4, pp. 555–596, 2008. [Online]. Available: https://aclanthology.org/J08-4004/

[30] B. Efron, "Bootstrap methods: Another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979. [Online]. Available: https://doi.org/10.1214/aos/1176344552

[31] R. Dror, G. Baumer, S. Shlomov, and R. Reichart, "The hitchhiker's guide to testing statistical significance in natural language processing," in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2018, pp. 1383–1392. [Online]. Available: https://aclanthology.org/P18-1128/

[32] Organisation for Economic Co-operation and Development, "Recommendation of the council on artificial intelligence," OECD Legal Instruments, Tech. Rep. OECD/LEGAL/0449, 2024, adopted in 2019 and amended in 2024. [Online]. Available: https://legalinstruments.oecd.org/en/instruments/OECD-LEGAL-0449

[33] Chính phủ Việt Nam, "Nghị định số 13/2023/nĐ-cp về bảo vệ dữ liệu cá nhân," Cổng Thông tin điện tử Chính phủ, Tech. Rep. 13/2023/NĐ-CP, 2023, ban hành ngày 17/04/2023, có hiệu lực từ 01/07/2023. [Online]. Available: https://chinhphu.vn/?classid=1&docid=207759&orggroupid=2&pageid=27160